IoT

MDPI

Article

G-PFL-ID: Graph-Driven Personalized Federated Learning for Unsupervised Intrusion Detection in Non-IID IoT Systems

Daniel Ayo Oladele *, Ayokunle Ige, Olatunbosun Agbo-Ajala, Olufisayo Ekundayo, Sree Ganesh Thottempudi, Malusi Sibiya and Ernest Mnkandla

Department of Computer Science, Florida Campus, University of South Africa, Johannesburg 1709, South Africa; agboaj@unisa.ac.za (O.A.-A.); ekundo@unisa.ac.za (O.E.); thotesg@unisa.ac.za (S.G.T.); sibiym@unisa.ac.za (M.S.); mnkane@unisa.ac.za (E.M.)

* Correspondence: oladeda@unisa.ac.za

Abstract

Intrusion detection in IoT networks is challenged by data heterogeneity, label scarcity, and privacy constraints. Traditional federated learning (FL) methods often assume IID data or require supervised labels, limiting their practicality. We propose G-PFL-ID, a graph-driven personalized federated learning framework for unsupervised intrusion detection in non-IID IoT systems. Our method trains a global graph encoder (GCN or GAE) with a DeepSVDD objective under a federated regularizer (FedReg) that combines proximal and variance penalties, then personalizes local models via a lightweight fine-tuning head. We evaluate G-PFL-ID on the IoT-23 (Mirai-based captures) and N-BaIoT (device-level dataset) under realistic heterogeneity (Dirichlet-based partitioning with concentration parameters $\alpha \in \{0.1, 0.5, \infty\}$ and client counts $K \in \{10, 15, 20\}$ for IoT-23, and natural device-based partitioning for N-BaIoT). G-PFL-ID outperforms global FL baselines and recent graph-based federated anomaly detectors, achieving up to 99.46% AUROC on IoT-23 and 97.74% AUROC on N-BaIoT. Ablation studies confirm that the proximal and variance penalties reduce inter-round drift and representation collapse, and that lightweight personalization recovers local sensitivity—especially for clients with limited data. Our work bridges graph-based anomaly detection with personalized FL for scalable, privacy-preserving IoT security.

Keywords: federated learning; graph neural networks; intrusion detection system; non-IID data; unsupervised learning; personalized models

1. Introduction

The Internet of Things (IoT) has shown many benefits in various fields, including healthcare [1], logistics, and smart infrastructure [2]. The IoT is an interconnected network of physical objects that can include everyday consumer electronics, smart home devices, enterprise edge gateways, and industrial infrastructure [3]. These devices use networked sensors and actuators to collect and analyze sensor data, improving operational efficiency, revealing system behavior, supporting predictive maintenance, and enabling new services [4–6]. However, the growth of interconnected devices increases the attack surface, opening up more possible attack points in the network.

Notable examples include botnets such as Mirai, Emotet, and Srizbi, which have exploited weaknesses in devices on a large scale [7–9]. Once compromised, these devices are often used for attacks such as denial of service (DoS), distributed denial of service (DDoS), and spoofing campaigns. These attacks exploit protocol or software vulnerabilities,

Check for updates

Academic Editor: Michele Nitti

Received: 15 December 2025
Revised: 14 January 2026
Accepted: 23 January 2026
Published: 29 January 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

overwhelm network or computing resources, impersonate legitimate traffic, and facilitate spamming, advertising fraud, intellectual property theft, ransomware deployment, and other fraudulent activities [10–12].

To reduce risks in IoT systems, operators typically deploy both preventive and detective controls. These techniques include network segmentation, hardened firewall rules, platform hardening, secure update mechanisms, and packet filtering appliances [13–17]. Although these techniques often serve as the first line of defense, blocking many known exploits, intrusion detection systems (IDS) remain crucial to monitoring network traffic and identifying attacks that evade these defenses [18].

Classical IDS approaches can be categorized as signature-based, anomaly-based, or hybrid-based. A signature-based IDS compares traffic with known threat patterns. Studies show that these techniques are effective at detecting familiar exploits with relatively few false alarms, yet they struggle with novel or zero-day attacks and require frequent updates [18–20]. Anomaly detectors (AD) learn patterns of expected behavior, such as normal traffic volume, protocol usage, and communication timing. They raise alerts when behavior diverges [21]. Once expected behaviors have been established, ADs are effective at identifying new threats; however, false alarm rates tend to increase unless baselines are carefully adjusted, particularly in dynamic IoT environments [18,20,21]. Hybrid systems combine methods to achieve better coverage and accuracy. However, in IoT deployments, this combination results in increased complexity, greater resource demands, and heavier maintenance burdens [18,20].

Machine learning (ML) expands the capabilities of IDS beyond handcrafted rules, enabling the detection of unfamiliar threats by learning directly from device logs [22]. However, supervised ML IDS requires labeled datasets to develop effective models. This process involves gathering and labeling large datasets that include current attack types across many different IoT devices. These datasets can be very expensive and often impractical, as discussed in [23–25]. A commonly proposed technique in the literature is training positive-only datasets to understand benign behavior and flag deviations [26,27].

Recent literature shows that graph-aware models, such as graph autoencoders (GAEs) and graph convolutional networks (GCNs), effectively detect anomalies in centralized settings when the data contains relational or structural dependencies. Network sensor data naturally records relationships between hosts and flows, such as sessions, services, ports, and device roles, as well as how these elements communicate with one another. Representing this data as a graph reveals multi-hop dependencies and structural outliers that pointwise methods often overlook [28–30]. However, deploying centralized ML-IDS in real IoT environments often poses practical challenges. For instance, collecting raw traffic to a central server increases communication overhead and latency. It also requires more computing power and memory than many edge devices can provide, creating a single point of failure [31,32].

Federated learning (FL) offers a promising solution to addressing these challenges. In this approach, each client trains using its own local data and only shares model updates with a coordinating server [33–37], which significantly reduces the transfer of raw data and provides stronger privacy protections. However, deploying FL in IoT environments presents unique challenges. Many IoT FL–based intrusion-detection studies assume only mild non-independent and identically distributed (non-IID) variation across clients. In practice, however, clients often differ substantially in the quantity and distribution of their data. Consequently, simple aggregation methods, such as FedAvg [38], can produce global models that perform poorly for clients with highly heterogeneous data distributions [36,39–43]

Additionally, embeddings may collapse during unsupervised or self-supervised graph training in FL if clients only observe very small local batches or experience intermittent

connectivity. Model aggregation becomes unstable when data is partitioned in a highly skewed manner, either by label or by volume. This instability is especially likely when many clients have only benign examples [44–47].

To address these issues, we propose a graph-based, personalized, unsupervised IDS trained in a federated manner across heterogeneous IoT clients. The approach follows a two-stage pipeline that begins with joint training of a shared graph encoder (a GCN or a GAE backbone) under a combined reconstruction and one-class objective [27]. Training is augmented with a proximal regularizer (FedProx) [48] and an embedding-variance penalty intended to stabilise updates when local data are scarce. The encoder is implemented in two complementary instantiations. The first is a graph-level GCN trained with a DeepSVDD objective that yields a compact hypersphere decision rule. The second is a hybrid GAE–DeepSVDD model that enforces reconstruction fidelity while also encouraging embedding compactness.

We adopt the hybrid variant as the canonical G-PFL-ID encoder because it combines the interpretability of reconstruction-based scoring with the explicit one-class boundary that operational detection benefits from. The GCN–DeepSVDD configuration is evaluated as a principled baseline and is used in ablations to probe how different inductive biases interact with the proposed federated regularizers and personalization choices.

After the global stage converges, clients perform light personalization by fine-tuning a small local head on benign examples and computing client specific calibration statistics. To simulate realistic heterogeneity we induce quantity skew via Dirichlet partitioning with concentration parameters $\alpha \in \{0.1, 0.5, \infty\}$ and we vary the number of clients in the federation $K \in \{10, 15, 20\}$. Empirical evaluation on IoT-23 under these controlled splits demonstrates consistent gains from the proposed regularizers and from local personalization.

The main contributions of this paper are as follows:

1. We implement and evaluate two federated one-class graph encoders that are compliant to federated training and comparable under identical conditions. These are a graph-level GCN trained with a DeepSVDD objective, and a hybrid GAE–DeepSVDD that jointly optimises reconstruction and embedding compactness.
2. We introduce two lightweight regularizers for federated unsupervised learning. A proximal aggregation term reduces weight divergence when client sample sizes vary. An embedding variance penalty mitigates representation collapse on small-data clients and preserves discriminative capacity.
3. We design a compact personalization head that each client fine-tunes on benign data. The head, together with selective encoder fine-tuning, restores local sensitivity while keeping communication overhead low.
4. We provide an extensive empirical study on IoT-23 using Dirichlet partitions with $\alpha \in \{0.1, 0.5, \infty\}$ and $K \in \{10, 15, 20\}$, and further validate G-PFL-ID on the N-BaIoT dataset (treating each of the 9 IoT devices as a separate client, $K = 9$) to demonstrate generalizability under real-world device heterogeneity. We report both global and personalized metrics (AUROC, AUPR, detection rates at fixed false-positive rates) and present ablation studies that isolate the contribution of each proposed component.

The rest of the paper is organized as follows. Section 2 situates our work relative to graph-based IDS and federated personalization. Section 3.1.1 formalizes the federated one-class problem and the partitioning protocols. Section 3 presents the model architecture, objectives, and federated algorithm. Section 4.1 details the experimental design and Section 4.2 reports the results and ablations. We conclude in Section 5 with limitations and directions for future work.

2. Related Works

In this section, we survey state-of-the-art work related to our proposal. We start with FL techniques applied to IoT intrusion or anomaly detection, then move to unsupervised anomaly detection methods, and finally discuss personalized federated learning approaches.

2.1. Federated Learning for Intrusion Detection

Existing studies have explored the use of federated learning for intrusion detection in various IoT and networked settings. Bhavsar et al. [49] proposed an edge-deployed federated learning intrusion detection system (FL-IDS) that runs lightweight classifiers on embedded devices and reported supervised training evaluated under an independent and identically distributed (IID) assumption on the NSL-KDD and Car Hacking datasets. Nguyen et al. [50] presented FedNIDS, a supervised, packet-level federated framework designed to disseminate incremental knowledge about new attacks. The paper reported strong empirical F1 scores, but the evaluation did not focus on extreme non-IID splits. Althunayyan et al. [51] designed a hierarchical federated learning (FL) system for in-vehicle Controller Area Network (CAN) bus monitoring. The system uses a two-stage detector with an LSTM autoencoder in the second stage. The proposed supervised model was evaluated on CAN datasets, but it did not explicitly stress-test severe client.

Huang et al. [52] introduced FED IoV, which converts vehicular traffic into images and trains MobileNet Tiny in a supervised, federated setting using the CAN Intrusion and CICIDS2017 datasets. The reported experiments assume independent and identically distributed (IID) clients and did not explore adversarial robustness. Shao et al. [53] used evolutionary neural architecture search to discover compact federated CNN models for industrial control systems. The supervised method was evaluated using the Gas Pipeline, SWaT, and WADI datasets, but non-IID behavior was not the main focus. Praharaj et al. [54] applied federated transfer learning to anomaly detection in cooperative smart farming. The authors reported supervised results on two smart farm testbeds, along with communication compression and acknowledge heterogeneity among farms but did not systematically parameterize it via Dirichlet or similar splits or measure the degree of heterogeneity. Singh and Popli [55] developed a supervised and validated federated IDS for underwater drones using a combination of in-house and public datasets, including CIC IDS 2017. However, the work did not deeply examine extreme non-IID participation.

Another group of studies looked at class imbalance and continuous learning, which are practical problems for supervised FL in IDS. Jin et al. [56] proposed FL-IDS, which augments local training with a class gradient balance loss and a relay client that fuses reconstructed samples. The evaluation was carried out using UNSW-NB15 and CIC-IDS17, showing improved resilience to catastrophic forgetting. However, the experiments consider moderate rather than extreme non-IID conditions. Mazid et al. [37] propose FL-IDPP, which used bidirectional RNNs locally and a voting ensemble at the server to improve decision quality. The paper reported reduced convergence time but does not analyze performance under strong client heterogeneity. Singh et al. [57] explicitly addressed class imbalance in federated IDS. The authors evaluated their class balancing strategies under both IID and moderate non-IID settings using the WUSTL IIoT 2021 dataset.

A common drawback of these studies is that they rely on supervised labels and therefore require access to representative attack annotations locally. Additionally, evaluations often assume mild heterogeneity or fail to systematically vary the degree of non-IID partitioning.

2.2. Unsupervised Anomaly Detection

Unsupervised techniques for anomaly detection in IoT networks do not require labeled attack examples, making them well-studied [21]. Classical methods include reconstruction

-based models, such as autoencoders [58] and variational autoencoders [59]. These models learn to reconstruct benign behavior and flag inputs with large reconstruction errors [26,28]. Other common families of methods include one-class and density estimators, such as Isolation Forest (IF) and Local Outlier Factor (LOF). These methods identify points as anomalous if they fall in low-density regions or are isolated from the main data mass [60]. Clustering and density-based approaches group normal instances and treat points belonging to very small or sparse clusters as anomalies [21]. These algorithms perform well in many settings but they face practical difficulties when applied to heterogeneous networked IoT data.

There are two relevant challenges in federated and constrained IoT deployments. First, reconstruction-based models typically rely on large, representative, benign datasets to learn reliable generative mappings. When clients have limited local data, the quality of reconstruction can degrade, raising both false negatives and false positives [61,62]. Second, unsupervised feature learners are prone to representation collapse under small sample sizes or small batches. Representation collapse occurs when encoders map distinct normal instances to overly similar latent vectors. This behavior has a negative effect on one-class detectors. For instance, DeepSVDD and related hypersphere methods rely on a compact but informative normality region. When embeddings collapse, the hypersphere becomes trivial, fails to separate anomalies from normal examples, and detection performance degrades [27,63].

Recent federated work proposes complementary solutions. Federated denoising autoencoders, such as L-MDAE, demonstrate that collaborative training across noisy, multimodal clients can enhance robustness [64]. Temporal hybrids combining LSTM encoders and autoencoders have been applied to grid and telemetry data in a federated manner, yielding promising results [65]. Other approaches share compressed distributional summaries or local density functions, using those summaries to guide contrastive or density-based learning at the client level [61,66]. Contrastive learning can increase embedding diversity, but it is conceptually at odds with one-class learning. The contrastive objective pushes many pairs apart, whereas the one-class objective prefers normal samples to remain close together. In federated non-IID settings, naive contrastive losses can amplify cross-client divergence unless coupled with careful regularization [61,67].

To address the limitations of naive contrastive losses, a family of density-based methods aims to preserve the geometry of high-density inlier subspaces. RIFIFI modifies the isolation forest paradigm to isolate outliers while preserving dense subspace structure. This improves interpretability and reduces false alarms [60]. However, these methods still neglect relational context, such as host-to-host relations or flow sequences, which are often essential for network intrusion detection [68,69].

Graph-based methods address this gap by using structural cues that single-point detectors cannot perceive. Graph autoencoders and their variational counterparts learn representations by reconstructing edges and node attributes. Anomalies are revealed via either large reconstruction error or embeddings that deviate from normal inlier [28,70,71]. Therefore, federated adaptations of graph learning have been proposed. For instance, FedGCN reduces communication per round by exchanging neighbor summaries beforehand and then updating local subgraphs [47].

Some frameworks apply contrastive self-supervision to graphs, maintaining global negative pairs or pseudo-labels to improve the separation between anomalous and normal nodes [72,73]. Methods such as LG-FGAD combine adversarial anomaly generation with local-global knowledge distillation to increase anomaly awareness in non-IID federated settings [74]. While these graph-aware federated methods often outperform tabular baselines on graph anomaly benchmarks, they introduce new system-level trade-offs. For exam-

ple, cross-client edge handling becomes necessary, per-round communication increases, and small local graphs become more sensitive to noise [68,75].

Deep one-class models, such as DeepSVDD, provide an explicit decision rule by learning a compact hypersphere that encloses normal data [27,76]. Combining reconstruction objectives with a one-class boundary (e.g., a VAE linked to SVDD) increases robustness because the model preserves data fidelity while enforcing a clearer decision surface [77,78]. These hybrids and pure one-class approaches are now being adapted to federated settings for both anomaly detection and defending against malicious updates [63]. However, applying naive federated averaging to DeepSVDD-style models can be fragile. When client embeddings drift inconsistently under severe non-IID sampling, the global hypersphere can be poorly defined, resulting in poor detection performance [79].

2.3. Personalized Federated Learning (PFL)

Personalization has emerged as a practical solution because a single global model rarely fits every client in a heterogeneous federation. In intrusion detection, for example, each site has its own mix of devices, traffic patterns, and operational constraints. A model tuned to these particularities often yields fewer false alarms, better local utility, and preserves the privacy benefits of federated training [80,81]. The empirical literature supports this intuition, and several distinct paradigms of personalization techniques have been developed.

One paradigm splits the model into a shared backbone and small client-specific modules, which could be a decoder or scoring head. The backbone is trained collaboratively across clients while the head remains private and is adapted locally. This pattern is easy to implement and keeps communication costs low because only the backbone parameters are exchanged during federation. Representative work includes FedPer [80] and FedRep [81], which demonstrate that separating shared representations from local heads can substantially improve performance for each client under heterogeneity. Ref. [82] also employed this paradigm to address the vulnerability of FL-based IDS systems to poisoned clients in non-IID IoT data environments. The proposed supervised PFL method for IDS (pFL-IDS) successfully defended against approximately ≈ 30 poisoned clients while maintaining a low attack success rate, outperforming prior approaches. However, detection is limited to the attacks on which it was trained.

Another family of methods uses proximal regularization or related penalties to limit how far local updates may deviate from the global solution. FedProx [48] is the canonical example, and Ditto [83], builds on this concept by framing personalization as a two-level problem. In this problem, each client learns a model that remains close to a global reference while optimizing its own local objective. Regularizers of this type are appealing in IDS because they explicitly balance global consensus with local specialization and allow for straightforward analysis and tuning via a single regularization coefficient.

Methods such as Per-FedAvg treat federated learning as a form of meta-training that produces an initialization that can be adapted with a few local gradient steps [84]. Meta-based personalization is attractive when clients have very few labeled examples or when fast local adaptation is required. In practice, this approach has been applied more often to supervised tasks. However, the underlying idea can be applied to one-class or unsupervised settings, where a shared encoder can be rapidly adapted to local normal behavior [84–86].

Another family of methods groups clients into clusters or learns a client similarity graph so that only similar participants share information with one another. Clustering reduces negative transfer from dissimilar peers and can be implemented in multiple ways. For example, it can be done by grouping gradient signatures [46], estimating pairwise

affinities [87], or learning a client network from marginal parameter statistics [88]. This family is relevant in graph-based federated anomaly detection because clients that host similar subgraphs or run similar services are natural sharing partners.

A notable trend regarding the benefits of these paradigms to FL is that they often improve worst-case performance per client rather than just average performance. This is important because a single poorly performing client can pose a significant risk [83]. However, two gaps are directly relevant to graph-based, unsupervised intrusion detection. First, many practical FL-IDSs operate on tabular features or time series and do not exploit the relational signals captured by graph models. Second, most personalization work has been developed independently of unsupervised, one-class objectives and the regularizer that stabilizes unsupervised, federated training. A summary of the existing related literature is presented in Table 1.

Table 1. Comparison of selected federated learning-based IDS works and the present paper.

Ref	Year	FL Algorithm	Local Model (Client)	Server Model/ Aggregation	Learning Type	Non-IID Reported	Personalized
Jin et al. [56]	2024	FedAvg with relay	CNN/CNN-GRU variants	Relay client fuses reconstructed samples	S	No	No
Mazid et al. [37]	2025	FedAvg + voting	Bi-RNN/RNN	Voting ensemble at server	S	No	No
Singh et al. [57]	2024	FedAvg variant (class-balanced)	DNN (local)	Aggregation with class-balancing adjustments	S	No	No
Nguyen et al. [50]	2025	FedAvg	Packet-level DNN	Aggregated DNN	S	No	FT
Xie et al. [46]	2021	Clustered FL	Local GNNs	Cluster-specific aggregations (gradient-based clustering)	S	Yes/JSD	C
Yao et al. [47]	2023	FedGCN (pre-exchange)	GCN variants (local)	Server aggregates precomputed neighbour summaries	SS	DD	No
Cai et al. [74]	2024	Custom federated GAD	GNN encoder + local discriminator	Dual knowledge distillation (local $\leftrightarrow$ global)	U	Yes/KL-D	KDist
Zhang et al. [63]	2025	Deep one-class in FL (defence)	DeepSVDD embeddings on local updates	Server filters suspicious updates using one-class scores	U	Yes/DPA	No
Sáez-de-Cámara et al. [87]	2023	Clustered FL + fingerprinting	Local unsupervised anomaly detectors	Server clusters devices by model fingerprints; cluster-wise training	U	Yes/Infer	C
Li et al. [83]	2021	Regularised FedAvg (bi-level optimisation)	Generic DNN models	Aggregates global model; each client solves personalised subproblem with proximal regulariser $\lambda\|\theta_i - \theta_g\|^2$	S	Yes/DP	RegP

Table 1. *Cont.*

Ref	Year	FL Algorithm	Local Model (Client)	Server Model/ Aggregation	Learning Type	Non-IID Reported	Personalized
Fallah et al. [84]	2020	Meta-learning based FL	Task-agnostic networks (MAML-style)	Global model provides initialisation optimised for fast client adaptation	S	Yes/TVD & 1-WD	MetL
Thein et al. [82]	2024	pFL-IDS (personalized)	Local supervised classifier	Aggregated backbone + local head	S	Yes/DP	FT
Ours	2025	FedProx variant + var-penalty	GCN/GAE backbone; small local one-class head	Aggregated backbone; local head personalisation & fine-tuning	U	Yes/(DD + KL-D)	FT & RegP

Columns report the following: the federated learning (FL) algorithm used, the local (client) model, and the server aggregation method. The learning type is indicated as supervised (S), semi-supervised (SS), or unsupervised (U). The Non-IID behaviour column specifies whether the model was tested under non-IID settings. The possible entries are: 'Not reported' (No), 'Mild' (mentioned but not experimentally reported), or 'Yes' (experimentally reported), with the metrics used for measurement listed. These metrics includes: Jensen-Shannon Distance (JSD), Dirichlet Distribution (DD), Generic Data partitioning (DP), Kullback–Leibler Divergence (KL-D), Direct Probabilistic Assignment (DPA), Total Variation Distance (TVD), 1-Wasserstein Distance (1-WD) and Device Characteristics (Infer). Finally, the Personalization column indicates whether personalization is provided. Personalization types include: Fine-Tuning Cluster (FT), Proximal Regularization (RegP), Meta-Learning (MetL), Clustering (C), Knowledge Distillation (Dist).

In this article, we combine these threads by federating a shared GNN encoder under a combined reconstruction and DeepSVDD one-class objective. We also augment local training with proximal and variance-preserving regularizers and allow lightweight local heads to personalize detection while minimizing backbone communication.

3. Methodology

3.1. Preliminaries

We present the preliminaries, which include the federated one-class problem for graph-structured IoT network traffic, and the definition of the main notation used for this study.

3.1.1. Problem Formulation

Given K clients with local datasets $\{\mathcal{D}_1, \mathcal{D}_2, \ldots, \mathcal{D}_K\}$ that contain primarily benign network traffic data, our goal is to learn a global model that can detect anomalous behavior while preserving data privacy. The objective is to minimize:

$$\min_{\theta} \; \mathcal{L}_{\text{global}}(\theta) \;=\; \sum_{k=1}^{K} \frac{n_k}{N} \mathcal{L}_k(\theta) \;+\; \Lambda(\theta), \tag{1}$$

where $\mathcal{L}_k(\theta)$ is the local unsupervised loss in client k and $\Lambda(\theta)$ denotes server-side regularization.

We transform network traffic into graph structures where anomalies manifest as deviations from learned normal patterns in the graph embedding space.

3.1.2. Notations and Definitions

We consider a federation of K clients (edge devices or gateways) indexed by $\mathcal{C} = \{1, \ldots, K\}$. Each client $k \in \mathcal{C}$ has a private data set $\mathcal{D}_k$ derived from network telemetry (Zeek conn logs in our experiments). From $\mathcal{D}_k$ we construct a client graph.

$$G_k = (V_k, E_k, X_k), \tag{2}$$

where V_k is the set of nodes (hosts), E_k the observed flow edges, and $X_k \in \mathbb{R}^{|V_k| \times d}$ the node feature matrix with d features per node. We denote by A_k the adjacency matrix of G_k and by $n_k = |V_k|$ the number of nodes on the client k. The total number of nodes in the federation is $N = \sum_{k=1}^{K} n_k$.

A shared graph encoder f_θ parameterized by θ maps a client graph to the node embeddings:

$$Z_k = f_\theta(X_k, A_k) \in \mathbb{R}^{n_k \times m}, \tag{3}$$

where each row $z_{k,i} \in \mathbb{R}^m$ is the embedding of node i and m is the embedding dimension.

Our pipeline supports three one-class instantiations: (i) GCN–DeepSVDD that directly optimizes a DeepSVDD hypersphere on graph-level GCN embeddings, (ii) GAE that uses reconstruction error on structure/features as an anomaly score, and (iii) a hybrid GAE–DeepSVDD that jointly optimizes reconstruction and hypersphere compactness. When using the GAE variant, the full model is f_θ (encoder) and g_ψ (decoder) and both sets of parameters can be updated during federated rounds to minimize reconstruction objectives. For DeepSVDD variants, the training focuses on f_θ and a one-class objective. The personalization parameters ϕ_k are introduced after the global backbone stabilizes. Clients initialize h_{ϕ_k} from the stabilized backbone provided by the server, then perform local fine-tuning on benign samples while keeping most of θ fixed or allowing limited encoder updates according to the selected personalization budget.

Table 2 summarizes the main symbols used throughout the paper.

Table 2. Notation summary.

Symbol	Meaning
K	Number of federated clients
$\mathcal{D}_k$	Local dataset at client k (flows, per-host aggregates)
$G_k = (V_k, E_k, X_k)$	Local graph at client k with nodes V_k, edges E_k, features $X_k \in \mathbb{R}^{n_k \times d}$
d	the input feature dimension (typically 128 for IoT-23 features)
n_k	Number of nodes at client k ($\|V_k\|$)
N	Total nodes across all clients, $N = \sum_k n_k$
f_θ	Shared graph encoder (GCN/GAE encoder) with encoder parameter θ shared globally via federation
g_ψ	Optional decoder for GAE reconstruction with decoder parameter ψ shared globally via federation
h_{ϕ_k}	Client k's lightweight personalization head
Z_k	Embeddings at client k, $Z_k = f_\theta(X_k, A_k) \in \mathbb{R}^{n_k \times m}$
m	Embedding dimension (typically 16 in our experiments)
$z_{k,i}$	Embedding for node i at client k (row of Z_k)
c / c_k	Global or client DeepSVDD center; c_k is client k's hypersphere center
μ	FedProx proximal coefficient used in local updates
λ	Weight decay regularization coefficient
α	Dirichlet concentration parameter for non-IID partitioning

3.2. G-PFL-ID Overview

The proposed Graph-based Personalized Federated Learning for Intrusion Detection (G-PFL-ID) framework addresses the fundamental challenges of IoT security in distributed environments through a comprehensive two-stage architecture. As illustrated in Figure 1, our approach combines global model federation with client-specific personalization to achieve robust generalization while maintaining individualized adaptation to local data distributions. The G-PFL-ID framework operates through the following systematic process:

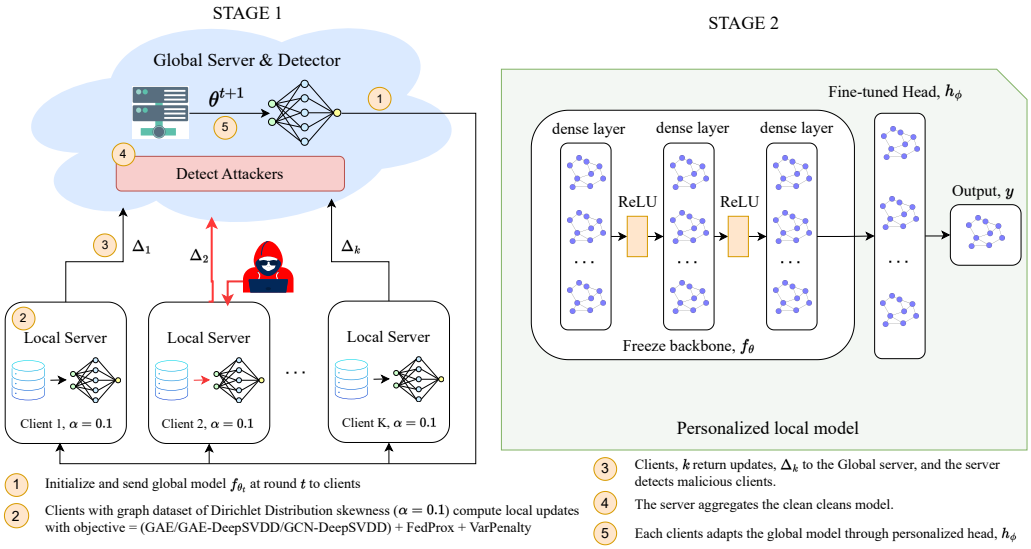


Figure 1. G-PFL-ID Framework Overview: Two-stage architecture combining global federated training with client-specific personalization. Stage 1 learns a shared global parameter θ from distributed client data. Clients run local one-class training with proximal and variance regularizers. Stage 2 adapts personalized parameter ϕ for individual clients.

- Global Model Initialization:The process begins with the central server initializing a shared global intrusion detection model f_θ. This model employs graph-based architectures which includes the option to use GCN encoders or GAE encoders with decoders g_ψ for reconstruction tasks [28,30,68]. The initialized model is broadcast to all participating clients at the beginning of each federation round t.
- Local Training with Multi-Objective Optimization: Each client k computes local updates using the shared global model f_θ on its per-host aggregated graph dataset $\mathcal{D}_k$. The local optimization incorporates three key components:
  - Task-Specific Objective: Clients employ one of three graph-based architectures: GAE (Graph Autoencoder), GAE-DeepSVDD, or GCN-DeepSVDD, depending on the detection paradigm (reconstruction-based or hypersphere-based anomaly detection).
  - FedProx Regularization: The proximal term $\frac{\mu}{2}\|\theta - \theta^t\|^2$ [48] stabilizes training under non-IID data distributions by constraining local updates to remain close to the global model.
  - Embedding Variance Penalty: A variance regularization term $\lambda_{\text{var}} \cdot \mathcal{L}_{\text{var}}(z)$ promotes compact feature representations, enhancing anomaly detection performance in one-class settings.
- Secure Server Aggregation with Intrusion Detection: The global server employs an intrusion detection mechanism to identify and filter out potentially malicious client updates Δ_k. Clean updates are aggregated using federated averaging [33]:

$$\theta^{t+1} \leftarrow \theta^t + \frac{1}{|\mathcal{S}_{\text{clean}}|} \sum_{k \in \mathcal{S}_{\text{clean}}} \Delta_k^t \tag{4}$$

  where $\mathcal{S}_{\text{clean}}$ represents the set of verified benign clients, θ^t and θ^{t+1} represents the global model parameters at round t and $t+1$ respectively, and Δ_k^t which is $\Delta_k^t = \theta_k^t - \theta^t$ represents the model update from client k at round t.
- Client-Specific Personalization: In the final stage, each client adapts the global model through personalized heads ϕ_k trained exclusively on local data $\mathcal{D}_k$. This personal-

ization phase optimizes ϕ_k^* ensuring optimal adaptation to local traffic patterns while maintaining stability [81,84].

$$\phi_k^* = \arg\min_{\phi_k} \mathbb{E}_{(x,y)\sim\mathcal{D}_k}[\ell(f_{\theta^*}(x) \circ \phi_k, y)] + \lambda_p \|\phi_k - \phi_{\text{init}}\|^2 \quad (5)$$

where ϕ_k^* represents the optimal personalized head for client k, $\arg\min_{\phi_k}$ finds parameters that minimize the objective, $\mathbb{E}_{(x,y)\sim\mathcal{D}_k}$ is the expectation over client k's data distribution, $\ell(\cdot, y)$ represents the loss function (e.g., reconstruction error for GAE), $f_{\theta^*}(x)$ is the feature representation from frozen global backbone, $\circ$ is the function composition (backbone + personal head), λ_p is the personalization regularization strength, and $|\phi_k - \phi_{\text{init}}|^2$ represents the L2 regularization from initial head.

3.3. Global Federated Training

The first stage of G-PFL-ID trains a shared graph encoder across the federation without centralizing raw network traffic. For a federation network containing K clients with local graph datasets $\{\mathcal{D}_k\}_{k=1}^K$, each client k provides a graph $G_k = (V_k, E_k, X_k)$ with $n_k = |V_k|$ nodes and node features $X_k \in \mathbb{R}^{n_k \times d}$. Our global backbone is a parameterised encoder f_θ that maps $(X_k, A_k) \mapsto Z_k \in \mathbb{R}^{n_k \times m}$.

We evaluate two principal one-class instantiations of f_θ: (i) *GCN–DeepSVDD* (encoder + one-class hypersphere objective) and (ii) *GAE–DeepSVDD* (encoder + decoder with joint reconstruction and one-class compactness). Figure 2 shows a schematic of the two alternatives, Table 3 presents the GCN–DeepSVDD architecture, and Table 4 presents the GAE-DeepSVDD architecture used in our experiments.

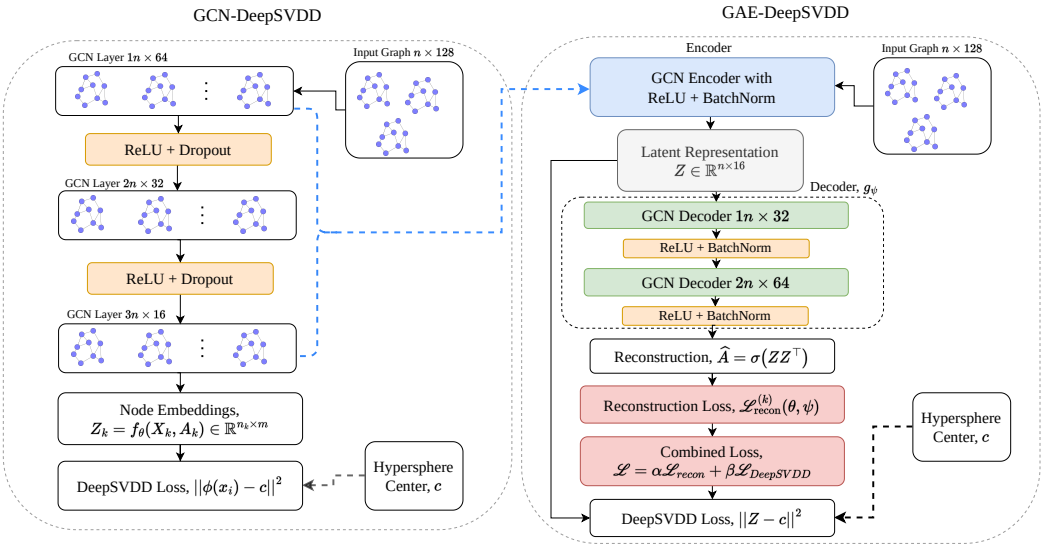


Figure 2. Architecture comparison: (**Left**) GCN-DeepSVDD maps input graphs to embeddings optimized to minimize distance from learned hypersphere center c. (**Right**) GAE-DeepSVDD encoder f_θ compresses graphs to latent representations while decoder g_ψ reconstructs adjacency matrix, with DeepSVDD simultaneously minimizing hypersphere distance.

3.3.1. GCN-DeepSVDD Implementation

In the GCN-DeepSVDD implementation, the hypersphere centre $c \in \mathbb{R}^m$ is initialized using a data-driven procedure [27] and remains fixed during training to prevent trivial solutions. The model optimizes the standard DeepSVDD objective:

$$\mathcal{L}_{\text{GCN-SVDD}}^{(k)}(\theta; \mathcal{D}_k) = \frac{1}{n_k} \sum_{i \in V_k} \|z_{k,i} - c\|^2 + \frac{\lambda}{2}\|\theta\|_F^2, \quad (6)$$

where $z_{k,i} \in \mathbb{R}^m$ represents the embedding of node i in client k which is also referred to as $\phi_\theta(x_i)$ in literature [27], λ is the weight decay coefficient, and $\|\cdot\|_F$ denotes the Frobenius norm. At inference, the anomaly score for node i is $s_{k,i} = \|z_{k,i} - c\|_2$.

Table 3. GCN –DeepSVDD architecture specifications.

Layer	Operation	Output Shape
Input	Node features $X \in \mathbb{R}^{n \times d}$	$n \times d$
GCN block 1	GCNConv($d \to 64$) $\to$ BatchNorm $\to$ ReLU	$n \times 64$
Dropout	$p = 0.3$	$n \times 64$
GCN block 2	GCNConv($64 \to 32$) $\to$ BatchNorm $\to$ ReLU	$n \times 32$
GCN block 3	GCNConv($32 \to m$) $\to$ Linear	$n \times m$
Embedding	Node representations $Z = f_\theta(X, A)$	$n \times m$
DeepSVDD	Euclidean distance to center $c \in \mathbb{R}^m$	$n \times 1$

Table 4. GAE–DeepSVDD architecture specifications.

Layer/Component	Operation	Output Shape
Input	Node features $X \in \mathbb{R}^{n \times d}$, adjacency A	$n \times d$, $n \times n$
Encoder block 1	GCNConv($d \to 64$) $\to$ BatchNorm $\to$ ReLU	$n \times 64$
Encoder block 2	GCNConv($64 \to 32$) $\to$ BatchNorm $\to$ ReLU	$n \times 32$
Encoder block 3	GCNConv($32 \to m$) $\to$ Linear	$n \times m$
Latent representation	$Z = f_\theta(X, A)$	$n \times m$
Decoder block 1	GCNConv($m \to 32$) $\to$ BatchNorm $\to$ ReLU	$n \times 32$
Decoder block 2	GCNConv($32 \to 64$) $\to$ BatchNorm $\to$ ReLU	$n \times 64$
Reconstruction	$\hat{A} = \sigma(ZZ^\top)$	$n \times n$
DeepSVDD	Euclidean distance to center $c \in \mathbb{R}^m$	$n \times 1$

3.3.2. GAE-DeepSVDD Implementation

The hybrid GAE-DeepSVDD model produces complementary anomaly signals through reconstruction error and embedding compactness. The reconstruction error detects local inconsistencies in graph structure, while embedding compactness enforces a tight one-class boundary, following principles established in graph autoencoder anomaly detection [89,90].

In our GAE-DeepSVDD implementation, the node features, $x_{k,i}$ are not reconstructed (i.e., $\hat{x}_{k,i} = 0$) for client k, only the adjacency matrix $\hat{A}$. Therefore the total hybrid loss, $\mathcal{L}^{(k)}_{\text{GAE-SVDD}}(\theta, \psi)$ for client k is expressed as:

$$\mathcal{L}^{(k)}_{\text{GAE-SVDD}}(\theta, \psi) = \alpha \mathcal{L}^{(k)}_{\text{recon}}(\theta, \psi) + \beta \mathcal{L}^{(k)}_{\text{SVDD}}(\theta) + \frac{\lambda}{2}\|\theta\|_F^2, \tag{7}$$

$$\mathcal{L}^{(k)}_{\text{recon}}(\theta, \psi) = \frac{1}{|\mathcal{E}_k|} \sum_{(i,j) \in \mathcal{E}_k} \ell_{\text{BCE}}(A_{ij}, \hat{A}_{ij}), \tag{8}$$

$$\mathcal{L}^{(k)}_{\text{SVDD}}(\theta) = \frac{1}{n_k} \sum_{i \in V_k} \|z_{k,i} - c\|_2^2, \quad z_{k,i} = f_\theta(X_k, A_k)[i]. \tag{9}$$

where:

- $\mathcal{L}^{(k)}_{\text{recon}}(\theta, \psi)$ and $\mathcal{L}^{(k)}_{\text{SVDD}}(\theta)$ are the graph reconstruction loss and DeepSVDD compactness loss for client k,
- α, and β are the reconstruction and DeepSVDD loss weight, where ($\alpha \in [0, 1]$), and ($\beta \in [0, 1]$).
- λ is the weight decay coefficient for $L2$ regularization
- $\|\theta\|_F^2$ is the frobenius norm of encoder parameters (sum of squared weights), and $\|\cdot\|_2^2$ is the squared $L2$ norm (Euclidean distance)
- $\ell_{\text{BCE}}(A_{ij}, \hat{A}_{ij})$ is binary cross-entropy between actual, A_{ij} and reconstructed, $\hat{A}_{ij}$ adjacency matrix element. $A_{ij} = 1$ if the edge (i, j) exists, and 0, otherwise. $\hat{A} = \sigma(ZZ^\top)$, σ is sigma,

- $(i, j) \in \mathcal{E}_k$ is the edge between nodes i and j which is an element of the edges, $\mathcal{E}_k$ in client k's graph.
- $x_{k,i}$, $\hat{x}_{k,i}$, and $z_{k,i}$ are the original node features, reconstructed node features, and latent embedding for node i in client k, where $z_{k,i} \in \mathbb{R}^m$.

Following the standard federated learning principles for autoencoders [33], where the complete model benefits from distributed learning of both encoding and decoding patterns across the federation, both the encoder parameters θ and decoder parameters ψ are updated locally and aggregated globally.

Since we only reconstruct the adjacency matrix (not node features, $x_{k,i}$), the reconstruction score is based solely on edge reconstruction errors. For node-level scoring, we aggregate edge errors. Therefore the final combined anomaly score, $s_{k,i}$ for node i in client k is expressed as:

$$s_{k,i} = \gamma s_{k,i}^{\text{SVDD}} + (1 - \gamma) s_{k,i}^{\text{rec}}. \tag{10}$$

where, $s_{k,i}^{\text{rec}} = \frac{1}{n_k} \sum_{j=1}^{n_k} \ell_{\text{BCE}}(A_{ij}, \hat{A}_{ij})$ represents the average reconstruction error or anomaly score of edges connected to node i, providing a node-centric reconstruction score, $s_{k,i}^{\text{SVDD}} = \|z_{k,i} - c\|_2$ represents the DeepSVDD anomaly score, and γ is the score fusion parameter (tuned on validation data, $\gamma \in [0, 1]$)

3.3.3. Federated Optimization with Multi-Objective Regularization (FedReg)

To address the challenges posed by non-IID data distributions and unstable convergence in federated one-class learning, we propose *FedReg*, a regularization framework that integrates multiple complementary objectives during local client training. For each client k, the local training objective, $\mathcal{L}_{\text{local}}^{(k)}$ combines three theoretically grounded regularization terms, as shown in Equations (14) and (15):

- Embedding Compactness, $\mathcal{L}_{\text{compact}}^{(k)}$: The variance regularization, $\text{Var}(Z_k)$, prevents feature collapse in unsupervised learning by encouraging diverse but structured representations [67,91]. This addresses the trivial solution problem where all embeddings converge to the hypersphere center c, which is crucial for effective anomaly detection in high-dimensional spaces [92].
- FedProx, $\mathcal{L}_{\text{prox}}^{(k)}$: Provides convergence guarantees for non-convex objectives under heterogeneous data distributions, with the proximal term bounding client drift by constraining local updates to remain close to the global model θ_G [48]. The hyperparameter μ controls the regularization strength.
- Adaptive Center Update, c: Ensures stable DeepSVDD optimization by gradually refining the hypersphere center, c while avoiding abrupt changes (model collapse) that could destabilize training [27].

$$\mathcal{L}_{\text{local}}^{(k)} = \mathcal{L}_{\text{task}}^{(k)} + \mathcal{L}_{\text{compact}}^{(k)} + \mathcal{L}_{\text{prox}}^{(k)} \tag{11}$$

$$\mathcal{L}_{\text{task}}^{(k)} = \begin{cases} \mathcal{L}_{\text{SVDD}}^{(k)} & \text{for GCN-DeepSVDD} \\ \alpha \mathcal{L}_{\text{recon}}^{(k)} + \beta \mathcal{L}_{\text{SVDD}}^{(k)} & \text{for GAE-DeepSVDD} \end{cases} \tag{12}$$

$$\mathcal{L}_{\text{compact}}^{(k)} = \lambda_{\text{compact}} \cdot \text{Var}(Z_k) = \lambda_{\text{compact}} \cdot \mathbb{E}[\|z - \mu_z\|^2] \tag{13}$$

$$\mathcal{L}_{\text{prox}}^{(k)} = \frac{\mu}{2} \|\theta_k - \theta_G\|^2 \tag{14}$$

where, $\mathcal{L}_{\text{task}}^{(k)}$ is the task-specific loss: The core detection objective varying by model architecture, as defined in Equations (6) and (7), $\mathcal{L}_{\text{compact}}^{(k)}$ is the embedding compactness, and $\mathcal{L}_{\text{prox}}^{(k)}$ is the FedProx regularization.

$$c \leftarrow (1-\alpha) \cdot c + \alpha \cdot \frac{1}{n_k} \sum_{i=1}^{n_k} z_{k,i} \tag{15}$$

where α is the update rate, and the center is initialized using a single forward pass on the first training batch of benign data. This approach follows established practices for stable DeepSVDD training [27]. Algorithm 1 provides the pseudo-code for the G-PFL-ID global federation training. During training, c is updated gradually using an exponential moving average (Equation (15)) to avoid sudden shifts that could destabilize the one-class objective. The update rate α is set to 0.1 in our experiments.

Algorithm 1 G-PFL-ID Federated Training with FedReg Regularization

Require: Client datasets $\{\mathcal{D}_k\}_{k=1}^{K}$, global model θ_G^0, FedReg hyperparameters
Ensure: Trained global model θ_G^T, personalized models $\{\theta_k^*\}$

Initialize global parameters θ_G^0, set round $t \leftarrow 0$
while $t < T$ and not converged **do**
  Sample client subset $\mathcal{S}^t \subset \{1, \ldots, K\}$ with fraction C
  for each client $k \in \mathcal{S}^t$ **in parallel do**
    $\theta_k^{t,0} \leftarrow \theta_G^t$ ▷ Initialize with global model
    Initialize optimizer with learning rate η, weight decay λ
    for local epoch $e = 1$ to E **do**
      for each batch b in $\mathcal{D}_k$ **do**
        Compute embeddings: $Z_k = f_{\theta_k}(X_b, A_b)$
        Compute task loss $\mathcal{L}_{\text{task}}^{(k)}$ via Equation (12)
        Compute compactness: $\mathcal{L}_{\text{compact}}^{(k)} = \lambda_{\text{compact}} \cdot \text{Var}(Z_k)$
        Compute FedProx: $\mathcal{L}_{\text{prox}}^{(k)} = \frac{\mu}{2} \|\theta_k - \theta_G^t\|^2$
        Total loss: $\mathcal{L}_{\text{local}}^{(k)} = \mathcal{L}_{\text{task}}^{(k)} + \mathcal{L}_{\text{compact}}^{(k)} + \mathcal{L}_{\text{prox}}^{(k)}$
        Update: $\theta_k \leftarrow \theta_k - \eta \nabla \mathcal{L}_{\text{local}}^{(k)}$
        if DeepSVDD model **then**
          Update center c via Equation (15) with rate α
        end if
      end for
    end for
    Upload local update: $\Delta_k^t = \theta_k - \theta_G^t$
  end for
  Server aggregates: $\theta_G^{t+1} \leftarrow \theta_G^t + \frac{1}{|\mathcal{S}^t|} \sum_{k \in \mathcal{S}^t} \Delta_k^t$
  $t \leftarrow t + 1$
end while
return θ_G^T, $\{\theta_k\}_{k=1}^{K}$ for personalization stage

Our global federated training operates under a strict one-class setting, where each client uses only its local benign data to train the shared graph encoder. This avoids the need for labeled attacks but assumes that the available benign data is representative of future normal traffic. The DeepSVDD objective enforces a compact hypersphere around these benign embeddings, which provides a clear decision boundary for anomaly detection.

3.4. Personalization Strategy

The second stage of G-PFL-ID performs client-specific adaptation to address statistical heterogeneity in federated IoT environments. Building on established personalized FL frameworks [93], it applies client-specific classification heads h_{ϕ_k} to tailor the globally trained model to each client's data distribution. Global model components remain frozen and the personalized heads are regularized, following [81,84], which helps maintain federation stability and preserve local data privacy.

For each client k, we optimize a personalized classification head, h_{ϕ_k} while keeping all global parameters fixed. Each client fine-tunes its head ϕ_k using only its local benign data. The loss is the same unsupervised objective used in global training plus a regularization term that discourages large deviations from the initial head:

$$\phi_k^* = \arg\min_{\phi_k} \mathcal{L}_{\text{personal}}^{(k)}(\phi_k; \theta_G, \psi_G, c) \tag{16}$$

$$\mathcal{L}_{\text{personal}}^{(k)} = \underbrace{\mathbb{E}_{x \sim \mathcal{D}_k^{\text{benign}}}\left[\ell\left(\phi_k \circ f_{\theta_G}(x)\right)\right]}_{\text{Unsupervised local loss}} + \underbrace{\lambda_p \|\phi_k - \phi_{\text{init}}\|^2}_{\text{Stability regularization}} \tag{17}$$

Here θ_G is the global encoder parameter (frozen), ψ_G is the global decoder parameter for GAE-DeepSVDD (frozen), c is the fixed DeepSVDD hypersphere center, λ_p is the personalization regularization strength, f_{θ_G} is the frozen global encoder backbone, θ_{init} is the initialized personal head, and ϕ_k represents the trainable client-specific personal head.

During inference, we combine global and personalized components:

$$\text{For GCN-DeepSVDD:} \quad s_{k,i} = \gamma \underbrace{\|f_{\theta_G}(x_i) - c\|_2}_{\text{Global SVDD}} + (1-\gamma) \underbrace{\phi_k(f_{\theta_G}(x_i))}_{\text{Personal head}} \tag{18}$$

$$\text{For GAE-DeepSVDD:} \quad s_{k,i} = \gamma_1 \underbrace{\|f_{\theta_G}(x_i) - c\|_2}_{\text{Global SVDD}} + \gamma_2 \underbrace{\|\hat{x}_i - x_i\|_2}_{\text{Global reconstruction}} + \gamma_3 \underbrace{\phi_k(f_{\theta_G}(x_i))}_{\text{Personal head}} \tag{19}$$

where $\sum \gamma_i = 1$ and weight γ (and $\gamma_1, \gamma_2, \gamma_3$ for GAE-DeepSVDD) is tuned on a held-out validation set of benign data from each client to balance the contribution of each anomaly signal.

Table 5 outlines the personalization strategy, including which parameters are frozen and which are adapted, to preserve global knowledge while allowing local client adaptation across both model variants. Algorithm 2 provides the pseudo-code for client personalization in G-PFL-ID.

Table 5. Personalization Strategy by Model Type.

Model Type	Frozen Components	Adapted Components
GCN-DeepSVDD	Early encoder layers (convs.0, projector.0-3)	Head layers, final encoder (convs.1, projector.4)
GAE-DeepSVDD	Early encoder layers	Decoder layers, final encoder layer

Algorithm 2 Client Personalization with Frozen Backbone

Require: Global encoder f_{θ^*}, client data $\mathcal{D}_k$, personalization epochs $E_{personal}$, personal head ϕ_k^0
Ensure: Personalized model ϕ_k^*
1: Initialize personal head ϕ_k^0 with shared initialization
2: **for** epoch $e = 1$ to E_{personal} **do**
3: Sample batch $\mathcal{B} \sim \mathcal{D}_k$
4: Compute embeddings: $Z = f_{\theta^*}(\mathcal{B})$ ▷ Frozen backbone
5: Compute anomaly score (e.g., $s = \|\phi_k(Z) - c\|_2$ for GCN-DeepSVDD) ▷ Trainable head
6: Compute loss: $\mathcal{L} = \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} s_i + \lambda_p \|\phi_k - \phi_k^0\|^2$
7: Update: $\phi_k \leftarrow \phi_k - \eta \nabla_{\phi_k} \mathcal{L}$
8: **end for**
9: **return** ϕ_k^*

4. Evaluation

This section describes the experimental setup used to evaluate the proposed G-PFL-ID framework, including dataset selection, preprocessing and graph construction, partitioning (non-IID) and analysis, baseline comparisons, and the evaluation protocol.

4.1. Experimental Setup

Table 6 shows the comprehensive hyperparameter specifications used in the implementation of the global federated training and personalized strategy stages.

Table 6. Comprehensive Hyperparameter Specifications.

Hyperparameter	Description	Typical Value
λ_{compact}	Embedding compactness weight	0.01
μ	FedProx regularization strength	0.01
K	Number of federated clients	10, 15, 30
α_c	Center update rate (DeepSVDD)	0.1
β	SVDD Weight	1.0
α	Dirichlet parameter for non-IID partitioning	$0.1, 0.5, \infty$
m	embedding Dim	16
-	Batch Size	64
ν	DeepSVDD hypersphere boundary parameter	0.1
η	Learning rate (Adam optimizer)	0.001
λ	Weight decay	1×10^{-4}
E	Local epochs	30
E_p	Personalization epochs	5
T	Number of server rounds (Global rounds)	20
rng	Random seeds	42

For N-BaIoT, we use the same hyperparameters except window size = 100 packets, graph type = temporal chain, and no Dirichlet partitioning (α not applicable).

We evaluated the proposed framework from both client and server perspectives using the Flower federated learning framework [94] and PyTorch [95] with PyTorch Geometric [96] for graph neural network operations. All experiments were conducted on a system with AMD Ryzen 9 5900X CPU (12 cores, 24 threads @ 3.70 GHz), 32 GB DDR4/3600 RAM, and an NVIDIA RTX 4090 GPU with 24GB VRAM. The software environment included Python 3.9, PyTorch 1.13.1, PyTorch Geometric 2.3.0, and Flower 1.4.0.

4.1.1. Dataset and Preprocessing

For computational efficiency, we selected `malware-capture-35-1` (Mirai botnet traffic) sub-capture containing 10,447,787 (8,262,389 benign and 2,185,398 malicious) connection records, each with 20 original Zeek connection log fields. This capture provides a realistic IoT attack scenario with sufficient data volume for meaningful federated learning experiments. The dataset supports both binary classification (benign vs. malicious) and one-class anomaly detection paradigms.

Zeek "conn" records were parsed using the header fields to extract connection attributes (timestamps, IPs, bytes, pkts, proto, service, conn_state, history, etc.). From raw fields we derived a richer set of per-flow features of a total of 48 as shown in Table 7. These features include: temporal (hour_of_day, day_of_week, is_weekend, log_duration, time_since_first), volumetric (orig/resp bytes & pkts, ip level bytes, missed bytes, bytes/packet ratios), connection state indicators (established, half-open, reset, etc.), protocol flags (SYN, ACK, FIN, RST, ICMP), and service one-hots (HTTP, DNS, DHCP, SSL, unknown). Categorical fields (e.g., `proto`, `service`, `conn_state`) were one-hot encoded and a $\log(1 + x)$ transform was applied to stabilize long-tailed numeric features (distributions).

We build per-host feature vectors by aggregating flow statistics for each unique IP address (using `id.orig_h` for source hosts and `id.resp_h` for destination hosts). For each host h, we compute separate source and destination feature aggregates:

- Source features: Mean values of numeric features when host h acts as source:

$$\text{src_mean}_h(x) = \frac{1}{N_{h,\text{src}}} \sum_{i:\text{src}_i=h} x_i \tag{20}$$

- Destination features: Mean values of numeric features when host h acts as destination:

$$\text{dst_mean}_h(x) = \frac{1}{N_{h,\text{dst}}} \sum_{i:\text{dst}_i=h} x_i \tag{21}$$

These source and destination feature sets are combined using an outer join, with missing values filled with zeros. The host label y_h is determined by logical OR aggregation across all flows involving host h:

$$y_h = \mathbb{1}[\exists \text{ flow } i : (\text{src}_i = h \vee \text{dst}_i = h) \wedge \text{label}_i = \text{malicious}] \tag{22}$$

In one-class mode, only benign flows were used for training, resulting in all host labels $y_h = 0$.

We construct an undirected graph $G = (V, E, X)$ where nodes V represent unique host IPs, and edges E connect hosts that communicate (at least one flow between them). Self-loops are removed. The final per-client graph is $G_k = (V_k, E_k, X_k)$ where $X_k \in \mathbb{R}^{|V_k| \times d}$ is the per-host feature matrix. To ensure consistent encoder input sizes, we pad or truncate feature vectors to a fixed dimension $d = 128$.

To evaluate the generalizability of G-PFL-ID, we also use the N-BaIoT dataset [97], which contains traffic from 9 real IoT devices infected by Mirai and BASHLITE botnets. Each device is treated as a separate client ($K = 9$), reflecting a natural (non-IID) partitioning because each device type has distinct traffic patterns. The dataset provides 115 statistical features extracted from traffic windows (e.g., packet counts, byte rates, jitter). We use only benign traffic for training and include all attack types in testing.

Since N-BaIoT does not contain explicit source/destination IPs, we construct a temporal graph for each device as follows. We split the traffic of each device into non-overlapping windows of 100 consecutive packets. For each window, we compute the mean of the 115 features, resulting in a node feature vector $x_i \in \mathbb{R}^{115}$. We then build an undirected chain graph connecting each window to its immediate predecessor and successor, capturing the temporal dependency of normal traffic. Self-loops are removed. The resulting graph for device k is $G_k = (V_k, E_k, X_k)$ where $|V_k|$ is the number of windows, E_k are the temporal edges, and $X_k \in \mathbb{R}^{|V_k| \times 115}$ is the node feature matrix. We standardize features per client and pad/truncate to a fixed dimension $d = 128$ for consistency with the encoder.

Table 7. Representative engineered per-flow/per-host features used to build node vectors (directional features are expanded to src/dst).

Feature Name	Description	Category
hour_of_day, day_of_week, is_weekend	Temporal time features to capture periodic patterns	Temporal
log_duration, time_since_first	Connection timing statistics	Temporal
orig_bytes, resp_bytes, orig_pkts, resp_pkts, orig_ip_bytes, resp_ip_bytes	Volumetric counts to identify traffic volume patterns from originator and responder	Volumetric
bytes_ratio(br), packets_ratio(pr), bytes_per_packet_orig(bp_{or}), bytes_per_packet_resp(bp_{res})	Ratios, $br = \frac{\text{orig_bytes}}{\text{resp_bytes}+\epsilon}$, $pr = \frac{\text{orig_pkts}}{\text{resp_pkts}+\epsilon}$, $bp_{or} = \frac{\text{orig_bytes}}{\text{orig_pkts}+\epsilon}$, $bp_{res} = \frac{\text{resp_bytes}}{\text{resp_pkts}+\epsilon}$	Volumetric

Table 7. *Cont.*

Feature Name	Description	Category
conn_state_*	One-hot connection states (established/reset/half-open/no-response/normal/other/rejected)	Connection State
proto_*	Protocol interactions (tcp/udp/icmp/other)	Protocol
syn, ack, fin, rst	TCP flags set	Protocol
init, data, timeout	Packet types and events	Protocol
syn_reply, ack_reply	Protocol response patterns	Protocol
handshake, fin_ack	Connection establishment/termination	Protocol
data_*	Data flow characteristics (direction/timeout)	Protocol
service_*	Service one-hots (http/dns/dhcp/ssl/unknown)	Service

* denotes categorical attributes that are transformed into one-hot encoded vectors prior to feature extraction.

We split each device's data into 70% benign for training, 10% benign for validation, and 20% (benign + all attacks) for testing. This construction allows us to test G-PFL-ID on a fundamentally different graph topology (temporal chains) compared to the host-based graphs of IoT-23, thereby assessing its adaptability.

4.1.2. Non-IID Partitioning and Client Splits

To simulate client heterogeneity, we partition the dataset across K clients using a label-dependent Dirichlet distribution, following established methodologies for federated learning benchmarking [98,99]. The partitioning process ensures that each client K receives a subset of data $\mathcal{D}_k$ with label distribution $p_k \sim \text{Dir}_K(\alpha \cdot \mathbf{1}_K)$, where $\alpha > 0$ controls the degree of heterogeneity.

We run experiments for $K \in \{10, 15, 20\}$ and $\alpha \in \{0.1, 0.5, \infty\}$ with smaller α (e.g 0.01 to 0.1) yielding highly skewed allocations, $\alpha = 0.5$ yielding medium skewness and large α (e.g ≥ 1) yielding near-uniform allocations i.e., IID. After obtaining the sample indices assigned to each client, we construct each client's local graph G_k using only its assigned flows (per-host aggregation). This yields client graphs that vary in size and connectivity, emulating realistic IoT edge heterogeneity.

Algorithm 3 summarizes the procedure used in our experiments. To avoid tiny clients that preclude local training, we enforce a minimum sample constraint min_samples, m (default $m = 100$). If the initial allocation yields some clients with fewer than m, we retry the sampling with a higher effective α (e.g., set $\alpha \leftarrow \max(\alpha, 1.0)$). For lower heterogeneity, α was set between 0.01 and 0.1; for medium heterogeneity, α was set between 0.3 and 0.6; and for IID partitioning, this was not an issue because α was set to 10^6. Additionally, we may increment the rng seed to introduce more randomness into the retry process. The best settings for α were 0.1, 0.5, and 10^6. Finally, the integer allocations are computed and adjusted to match the total remaining samples.

Once hosts are assigned to clients, each client's host set is partitioned into train/validation/test splits (e.g., 70/10/20). All experiments are controlled by a single experiment seed ($rng = 42$).

To quantify the degree of non-IIDness across client partitions, we employ multiple complementary metrics that capture different aspects of distributional heterogeneity. We utilize Jensen-Shannon (JS) divergence as our primary metric for quantifying distributional differences between clients and the global dataset. The JS divergence provides a symmetric and bounded measure of distribution similarity, making it ideal for federated learning scenarios [100]. It ranges from 0 (identical distributions) to 1 (completely dissimilar distributions), with values below 0.1 typically indicating good distributional alignment. For a client distribution P_k and global distribution P_{global}, the JS divergence is defined as:

$$\mathrm{JS}(P_k \| P_{\text{global}}) = \frac{1}{2} D_{\mathrm{KL}}(P_k \| M) + \frac{1}{2} D_{\mathrm{KL}}\left(P_{\text{global}} \middle\| M\right) \tag{23}$$

where $M = \frac{1}{2}(P_k + P_{\text{global}})$ is the mixture distribution, and D_{KL} denotes the Kullback-Leibler divergence:

$$D_{\mathrm{KL}}(P \| Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)} \tag{24}$$

where $P(x)$ represents the probability distribution of client k, and $Q(x)$ is the global probability distribution across all clients.

Algorithm 3 Dirichlet-based Non-IID Partitioning with Minimum Sample Guarantees

Require: Labels $\mathbf{y} \in \mathbb{R}^N$, client count K, concentration α, minimum samples m, random seed
Ensure: Client indices $\{\mathcal{I}_1, \ldots, \mathcal{I}_K\}$
Initialize $\mathcal{I}_k \leftarrow \emptyset$ for $k = 1, \ldots, K$
$\mathbf{y} \leftarrow \text{np.array}(\mathbf{y})$, $rng \leftarrow \text{RandomState(seed)}$
$classes, counts \leftarrow \text{unique}(\mathbf{y}, \text{return_counts} = \text{True})$
if $|classes| = 1$ **then** ▷ Single-class scenario
    $\mathcal{I} \leftarrow \text{arange}(N)$; $rng.\text{shuffle}(\mathcal{I})$
    assert $N \geq K \cdot m$ ▷ Minimum sample requirement
    Assign first $K \cdot m$ samples: $\mathcal{I}_k \leftarrow \mathcal{I}[(k-1)m : km]$ for $k = 1, \ldots, K$
    $\mathcal{I}_{\text{rem}} \leftarrow \mathcal{I}[K \cdot m :]$ ▷ Remaining samples
    $\mathbf{p} \leftarrow rng.\text{dirichlet}([\alpha] \times K)$ ▷ Skew proportions
    $\mathbf{c} \leftarrow \text{floor}(\mathbf{p} \cdot |\mathcal{I}_{\text{rem}}|)$ ▷ Integer allocations
    Adjust $\mathbf{c}$ to satisfy $\sum_k c_k = |\mathcal{I}_{\text{rem}}|$
    Assign $\mathcal{I}_{\text{rem}}$ proportionally to $\mathbf{c}$
end if
if $\min_k |\mathcal{I}_k| < m$ **then** ▷ Minimum sample validation
    Recursively retry with increased $\alpha \leftarrow \max(\alpha, 1.0)$
end if
return $\{\mathcal{I}_1, \ldots, \mathcal{I}_K\}$

Complementary to JS divergence, we measure the spread of feature values within each client called the `feature variances`. A high variation in feature variance across clients indicates `heteroscedasticity`.

Figure 3 presents the comprehensive non-IID analysis across three Dirichlet concentration parameters. The high non-IID setting ($\alpha = 0.1$) exhibited extreme heterogeneity with JS divergences ranging from 0.011 to 0.405, the moderate non-IID setting ($\alpha = 0.5$) showed significantly reduced heterogeneity with JS divergences tightly clustered around 8.2×10^{-5}. The same applied to the feature variances. The near-IID setting ($\alpha \to \infty$) approached homogeneous distribution with JS divergences nearly identical across client, ($\sim 1.2 \times 10^{-5}$) and feature variances showing minimal variation.

The N-BaIoT dataset provides a natural non-IID partitioning where each IoT device represents a distinct client with inherently different traffic characteristics. We quantify this heterogeneity using three complementary metrics computed on benign samples (Table 8):

- Quantity skew. Data volume varies substantially across devices. In our benign subset the largest device (Device 4: Philips baby monitor) contributes 175,240 samples while the smallest device (Device 2: Ecobee thermostat) contributes 13,113 samples, a max/min ratio of approximately 13.4:1. This degree of skew reflects real-world deployments where device types and usage patterns differ.

- Gini coefficient. The Gini coefficient for the nine-device benign split is $G \approx 0.37$, indicating a moderate level of inequality in sample counts (with a small number of devices contributing a large fraction of the data). We compute G as

$$G = \frac{\sum_{i=1}^{n} \sum_{j=1}^{n} |x_i - x_j|}{2n \sum_{i=1}^{n} x_i} = 0.37, \quad (25)$$

  where x_i denotes the benign-sample count of device i and $n = 9$.
- Coefficient of variation (CV). The mean number of benign samples per device is $\bar{x} = 61{,}770$ and the standard deviation is $\sigma \approx 46{,}385$, yielding CV $= \sigma/\bar{x} \approx 0.75$. This high relative variability further establishes the imbalance that federated algorithms must tolerate during aggregation and personalization.

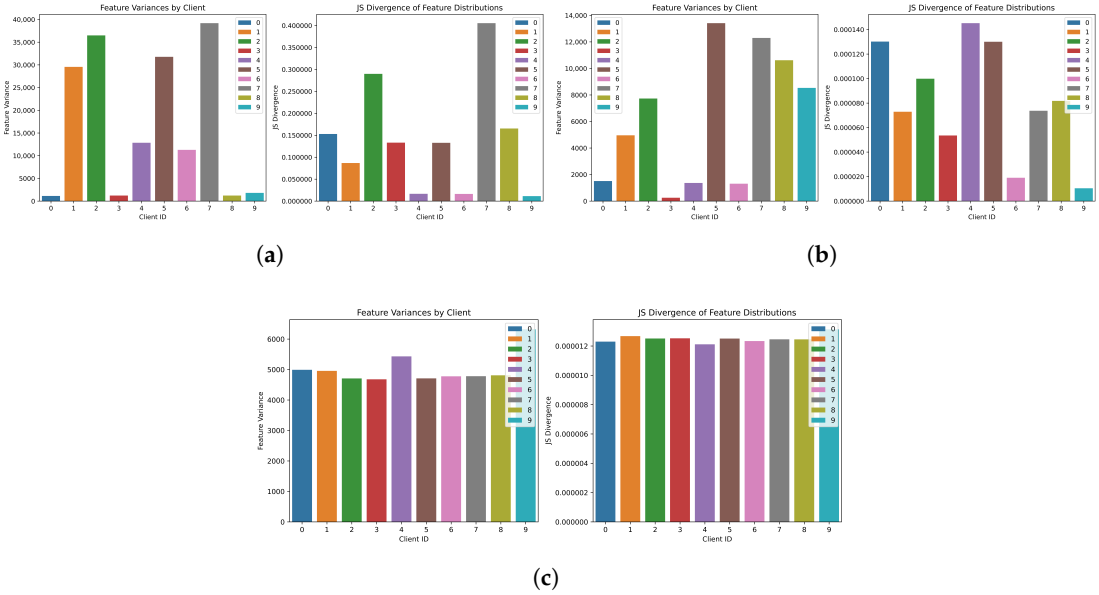


Figure 3. Comprehensive non-IID analysis across heterogeneity levels. The first graph shows the Feature variance distributions across clients, and the second Jensen-Shannon divergence between client and global distributions. Figure (**a**) is the non-IID analysis for $\alpha = 0.1$ (**b**) non-IID analysis for $\alpha = 0.5$, and (**c**) non-IID analysis for $\alpha = \infty$. Results demonstrate progressive distributional alignment from high non-IID ($\alpha = 0.1$) to near-IID ($\alpha = \infty$) conditions, with significant heterogeneity in both label distributions and feature characteristics under realistic IoT scenarios.

Table 8. Benign-sample counts and heterogeneity statistics for N-BaIoT devices (used as clients).

Device ID	Device Type	Manufacturer	City	Country	Benign Samples
1	Danmini Doorbell	Unknown/OEM	N/A	N/A	49,548
2	Ecobee Thermostat	ecobee Inc.	Toronto	Canada	13,113
3	Ennio Doorbell	Unknown/OEM	N/A	N/A	39,100
4	Philips Baby Monitor	Koninklijke Philips N.V.	Amsterdam	Netherlands	175,240
5	Provision PT-737E Camera	Provision-ISR	Kfar Saba	Israel	62,154
6	Provision PT-838 Camera	Provision-ISR	Kfar Saba	Israel	98,514
7	Samsung Webcam	Samsung Electronics	Suwon	South Korea	52,150
8	SimpleHome XCS7-1002 Camera	Unknown/OEM	N/A	N/A	46,585
9	SimpleHome XCS7-1003 Camera	Unknown/OEM	N/A	N/A	19,528
Mean	–	–	–	–	**61,770**
Std. dev.	–	–	–	–	**46,385**
CV	–	–	–	–	**0.75**
Gini	–	–	–	–	**0.37**
Max/Min ratio	–	–	–	–	**13.4:1**

Notes: statistics computed on benign sample counts shown above. Manufacturer/city/country information is included where publicly available; some entries correspond to OEM or brand names without documented corporate headquarters. Mean and standard deviation are rounded to the nearest integer; CV and Gini are shown with two significant digits.

4.2. Discussion and Results

To demonstrate the various strengths of the proposed G-PFL-ID framework, we conducted and report experiments under two different settings: (1) core anomaly detection capability under the moderate non-iid setting $\alpha = 0.5$, and (2) the contribution of the regularizers and personalization the under the highest non-iid setting, and (3) the G-PFL-ID frameworks performance across all clients and α settings (ablation) is presented.

4.2.1. Experimental Setting 1: Global Anomaly Detection Performance

Table 9 presents the primary detection results under representative heterogeneity settings for both IoT-23 ($\alpha = 0.5$, $K = 10$) and N-BaIoT ($K = 9$) datasets. We compare classical one-class algorithms (Isolation Forest, One-Class SVM), the recent graph-based federated anomaly detection method LG-FGAD [74], federated MLP baselines (FedAvg and FedProx), and our proposed federated graph methods (GCN-DeepSVDD and GAE-DeepSVDD). The table reports area under the ROC curve (AUROC), area under the precision-recall curve (AUPR), true positive rates at false positive rates of 1%, 5%, and 10%, and inference time per sample.

The hybrid GAE-DeepSVDD consistently achieves the best detection performance across both datasets. On IoT-23, it yields 99.46% AUROC and 99.94% AUPR, substantially outperforming all baselines including LG-FGAD (96.10% AUROC). This performance advantage is pronounced at low false positive rates: at 1% FPR, GAE-DeepSVDD achieves 80.16% TPR compared to LG-FGAD's 42.50% TPR—nearly double the detection rate. These results suggest that compared to the LG-FGAD's adversarial augmentation approach, under moderate non-IID conditions, the hybrid objective (combining reconstruction and compactness) produces effective discriminative embeddings where anomalous nodes are well separated from normal ones, enabling high detection sensitivity while keeping false alarms very low.

The results also shows that the graph-based methods consistently outperform traditional feature-based approaches, confirming that graph encoders capture relational patterns—host homophily, service dependencies, temporal correlations—that flat feature representations miss. For instance, the pure GCN-DeepSVDD (97.25% AUROC on IoT-23) exceeds all non-graph baselines, including federated MLP variants. Even on N-BaIoT, where we construct simple temporal chain graphs, GCN-DeepSVDD achieves 96.48% AUROC, demonstrating that minimal graph structure provides valuable inductive bias.

Table 9. Global anomaly detection performance on IoT-23 (host-based graph, $\alpha = 0.5$, $K = 10$) and N-BaIoT (9 device-clients). Baselines (rows 1–5) are evaluated on IoT-23; our graph methods (rows 6–9) are evaluated on both datasets. Best mean per column is **bold**.

Method	AUC-ROC	AUPR	TPR@1%FPR	TPR@5%FPR	TPR@10%FPR	Infer. Time (ms)
Isolation Forest	91.25 ± 1.59	94.47 ± 0.45	0.62 ± 0.44	24.43 ± 1.39	53.47 ± 5.76	2.586 ± 0.39
One-Class SVM	78.56 ± 1.03	91.16 ± 0.79	2.47 ± 1.66	16.81 ± 6.25	38.92 ± 4.93	4.571 ± 0.68
LG-FGAD [74]	96.10 ± 1.10	97.10 ± 0.70	42.50 ± 5.23	66.30 ± 3.64	92.80 ± 2.11	2.187 ± 0.20
FedAvg + MLP	93.92 ± 1.26	96.13 ± 0.11	10.02 ± 4.84	50.02 ± 1.52	91.19 ± 0.25	3.899 ± 0.32
FedProx + MLP	94.35 ± 2.01	98.27 ± 0.47	15.33 ± 2.03	53.94 ± 1.30	93.19 ± 0.56	3.133 ± 0.03
N-BaIoT dataset ($K = 9$)						
GCN–DeepSVDD	96.48 ± 0.89	96.89 ± 0.67	45.67 ± 4.12	68.90 ± 2.45	93.67 ± 1.23	**1.234 ± 0.12**
GAE–DeepSVDD	97.74 ± 0.45	98.28 ± 0.23	65.89 ± 3.78	86.34 ± 1.89	97.92 ± 0.25	1.567 ± 0.15
IoT-23 Dataset ($\alpha = 0.5, K = 10$)						
GCN-DeepSVDD	97.25 ± 1.18	98.82 ± 0.76	23.27 ± 4.13	61.59 ± 3.05	94.62 ± 0.89	1.553 ± 0.17
GAE-DeepSVDD	**99.46 ± 0.27**	**99.94 ± 0.02**	**80.16 ± 5.87**	**93.83 ± 0.47**	**100.00 ± 0.00**	1.989 ± 0.23

Inference times per sample remain practical for real-time deployment. GCN-DeepSVDD is fastest at 1.55 ms per inference on IoT-23, while the hybrid GAE-DeepSVDD requires 1.99 ms—both suitable for near-real-time detection on IoT gateways. LG-FGAD requires 2.19 ms, slightly slower despite comparable graph processing. The modest computational overhead of our hybrid model is justified by its significantly better detection performance, particularly at stringent operating points.

Despite extreme natural heterogeneity (9.0:1 quantity skew, Gini coefficient 0.37) of the N-BaIoT, our methods maintain strong performance: GAE-DeepSVDD achieves 97.74% AUROC and 98.28% AUPR. This demonstrates that G-PFL-ID generalizes beyond host-based graphs to temporal constructions, and handles real-world device heterogeneity effectively. The performance gap between GAE-DeepSVDD and GCN-DeepSVDD narrows slightly on N-BaIoT (1.26 percentage points versus 2.21 points on IoT-23), suggesting that the reconstruction objective provides less advantage when graph structure is simpler. Nonetheless, both variants significantly outperform classical and federated MLP baselines.

These findings validate our core design principles. The hybrid reconstruction and compactness objective produces more discriminative embeddings than pure one-class or adversarial approaches. Graph encoders, even with simple constructions, capture relational patterns that boost detection accuracy. And the framework scales effectively from artificially partitioned datasets (IoT-23) to naturally heterogeneous deployments (N-BaIoT), maintaining strong performance under diverse operational conditions.

4.2.2. Experimental Setting 2: Component Contribution Analysis

Table 10 reports per-client AUROC under three configurations: (i) the base model alone (no federated regularizer, no personalization), (ii) with federated regularizers (FedProx proximal term plus an embedding variance penalty), and (iii) with personalization (a lightweight local head fine-tuned on each client plus center recalibration). Results are shown for three degrees of heterogeneity: high non-IID ($\alpha = 0.1$), medium non-IID ($\alpha = 0.5$), and near-IID ($\alpha \to \infty$).

Across all heterogeneity regimes, adding FedReg yields a clear and consistent improvement in average per-client AUROC while reducing variance. For instance, with GCN DeepSVDD at $\alpha = 0.1$ the mean client AUROC rises from about 88.3% for the base model to 92.6% with FedReg, and for GAE DeepSVDD it rises from 91.0% to 96.1%. These gains are most pronounced in the highly non-IID setting, which matches our design intuition. The FedProx term limits large weight drift under quantity and distribution skew, and the variance penalty prevents each client's representations from collapsing when its data are sparse. This is reflected in the reduced standard deviations across clients after FedReg, indicating more consistent performance.

The natural heterogeneity of N-BaIoT presents a distinct challenge with a 9.0:1 quantity skew across devices (Table 8). Despite this extreme imbalance, FedReg provides substantial improvements, raising GAE-DeepSVDD performance from 93.48% to 95.55% on average. Notably, low-data devices (e.g., Device 2 with 13,113 samples) benefit the most, with AUROC improvements of 3-4 percentage points. This demonstrates FedReg's effectiveness in real-world scenarios where device heterogeneity is intrinsic rather than artificially induced.

Local personalization produces a further and often substantial uplift in per-client AUROC. Continuing the previous example (GCN DeepSVDD at $\alpha = 0.1$), personalization increases the mean AUROC from 92.6% (with FedReg) to 97.0%. Crucially, personalization brings up the weaker clients that suffer the most from distribution mismatch. After personalization, clients that were previously in the low 80s rise into the mid 90s. In other words, adding a small local head and recalibrating the center effectively recovers local sensitivity without retraining the entire encoder or heavy communication.

On N-BaIoT, personalization yields an additional 2.19 percentage point improvement for GAE-DeepSVDD (from 95.55% to 97.74%), with low-data devices showing the largest gains. Device 2, for instance, improves from 90.33% to 93.10%, demonstrating that even with extreme quantity skew, lightweight personalization can significantly enhance local detection performance. This is particularly valuable in IoT deployments where devices cannot share their raw data but can afford to fine-tune a small local module.

Over all α settings, the hybrid GAE model achieves slightly higher average AUROC and smaller variance after FedReg and personalization compared to the pure GCN. For example, in the high non-IID case the GAE DeepSVDD reaches 97.21% average after personalization versus 97.00% for the GCN variant. We attribute this to the complementary learning objectives: the reconstruction term in the GAE offers a strong local constraint that is less affected by moderate distribution shifts, while the SVDD term aligns representations globally. Together they stabilize learning under heterogeneity.

A key concern in federated learning is poor worst-case performance on some clients. The per-client results in Table 10 show that the worst-performing clients under the base model improve markedly with FedReg and personalization. For instance, at $\alpha = 0.1$ the lowest GCN DeepSVDD client AUROC moves from the low 80s in the base model to the mid-90s after personalization. This shows that our approach not only raises the average performance but also reduces client-level risk, addressing an important gap emphasized in personalized FL literature [83].

These findings illustrate several important points. First, relational inductive bias matters: graph encoders exploit patterns such as host homophily and repeated service patterns that simple feature models miss, yielding higher overall AUROC and especially much better detection at low FPR (which is critical in IDS). Second, the hybrid reconstruction-plus-SVDD objective works particularly well: the autoencoder forces the model to preserve local graph topology, while the SVDD term carves out a tight hypersphere of normal instances. Together they produce embeddings where anomalies fall clearly outside the sphere, in line with findings in [77]. Third, FedReg (FedProx plus variance penalty) effectively curbs model drift and collapse on clients with few samples, as discussed in [48,67]. Finally, personalization through a local head and center calibration leverages each client's own benign data distribution to fine-tune thresholds without heavy overhead, significantly improving per-client sensitivity. This confirms that modest personalization can yield large practical gains in unsupervised one-class federated learning.

Table 10. G-PFL-ID per-client AUC-ROC (%) results under different non-IID settings.

Client ID	GCN-DeepSVDD			GAE-DeepSVDD		
	Base-Model Only	**w/FedReg**	**w/Personalization**	**Base-Model Only**	**w/FedReg**	**w/Personalization**
	High non-IIDness ($\alpha = 0.1$)					
Client 0	85.66 ± 3.25	92.00 ± 0.96	95.97 ± 1.05	90.89 ± 3.52	95.78 ± 0.87	96.58 ± 0.58
Client 1	87.57 ± 2.90	91.91 ± 1.03	96.68 ± 0.47	89.80 ± 2.81	95.09 ± 0.94	96.49 ± 0.46
Client 2	84.59 ± 6.15	89.93 ± 4.84	94.30 ± 1.20	88.32 ± 4.84	93.41 ± 1.62	94.51 ± 0.98
Client 3	89.72 ± 1.33	93.06 ± 0.49	97.43 ± 0.59	91.45 ± 2.14	96.54 ± 0.71	97.64 ± 0.42
Client 4	91.37 ± 1.19	94.71 ± 0.37	99.08 ± 0.33	93.10 ± 0.79	98.19 ± 0.62	99.29 ± 0.09
Client 5	89.81 ± 4.90	94.15 ± 0.17	98.52 ± 0.34	92.54 ± 1.12	97.63 ± 0.50	98.73 ± 0.25
Client 6	92.77 ± 0.13	95.11 ± 0.28	99.48 ± 0.03	93.50 ± 0.09	98.59 ± 0.10	99.69 ± 0.07
Client 7	86.07 ± 3.20	90.41 ± 1.92	95.78 ± 0.91	88.80 ± 5.76	93.89 ± 1.30	94.99 ± 1.19
Client 8	84.51 ± 5.87	89.85 ± 2.48	93.22 ± 2.03	88.24 ± 5.14	93.33 ± 1.66	94.43 ± 0.79
Client 9	90.85 ± 0.09	95.19 ± 0.08	99.56 ± 0.07	93.58 ± 0.03	98.67 ± 0.09	99.77 ± 0.05
Average	88.29 ± 2.90	92.63 ± 1.26	97.00 ± 0.70	91.02 ± 2.62	96.11 ± 0.84	**97.21 ± 0.49**

Table 10. *Cont.*

Client ID	GCN-DeepSVDD			GAE-DeepSVDD		
	Base-Model Only	**w/FedReg**	**w/Personalization**	**Base-Model Only**	**w/FedReg**	**w/Personalization**
			Medium non-IIDness ($\alpha = 0.5$)			
Client 0	91.98 ± 1.30	94.53 ± 0.64	96.28 ± 0.62	93.89 ± 2.92	95.88 ± 0.62	99.06 ± 0.79
Client 1	94.90 ± 0.42	97.09 ± 0.49	97.45 ± 0.73	96.94 ± 0.10	97.57 ± 0.08	99.66 ± 0.17
Client 2	96.24 ± 0.50	95.06 ± 0.54	96.02 ± 0.56	94.48 ± 0.51	97.14 ± 0.09	99.49 ± 0.11
Client 3	97.96 ± 0.29	97.81 ± 0.57	98.26 ± 0.38	96.38 ± 0.09	99.09 ± 0.11	100.00 ± 0.00
Client 4	89.74 ± 2.84	90.71 ± 1.15	94.82 ± 0.46	90.91 ± 0.12	95.19 ± 0.86	98.93 ± 1.06
Client 5	91.38 ± 0.35	93.93 ± 0.64	95.78 ± 0.39	91.12 ± 0.21	95.41 ± 0.70	99.14 ± 0.33
Client 6	98.02 ± 0.18	98.25 ± 0.08	98.96 ± 0.07	97.06 ± 0.01	99.53 ± 0.12	100.00 ± 0.00
Client 7	94.00 ± 1.04	97.49 ± 0.25	97.49 ± 0.40	94.94 ± 2.10	97.87 ± 0.68	99.86 ± 0.09
Client 8	94.88 ± 0.83	96.78 ± 0.31	98.05 ± 0.32	94.92 ± 0.10	97.16 ± 0.10	99.46 ± 0.34
Client 9	99.04 ± 0.07	99.16 ± 0.00	99.38 ± 0.07	98.58 ± 0.06	99.94 ± 0.13	100.00 ± 0.00
Average	94.88 ± 0.78	96.08 ± 0.47	97.25 ± 0.40	94.92 ± 0.62	97.48 ± 0.35	**99.56 ± 0.29**
			Low non-IIDness ($\alpha = \infty$)			
Client 0	96.46 ± 1.05	97.22 ± 0.12	99.95 ± 0.01	97.08 ± 0.18	100.00 ± 0.00	100.00 ± 0.00
Client 1	96.79 ± 1.20	97.95 ± 0.10	98.19 ± 0.09	97.73 ± 0.15	97.74 ± 0.91	99.98 ± 0.01
Client 2	95.77 ± 3.45	99.10 ± 0.25	99.49 ± 0.15	96.95 ± 0.35	98.53 ± 0.41	100.00 ± 0.00
Client 3	96.56 ± 0.30	98.03 ± 0.15	98.36 ± 0.46	97.27 ± 0.22	98.60 ± 0.29	99.97 ± 0.01
Client 4	97.14 ± 0.95	96.49 ± 2.08	100.00 ± 0.00	98.43 ± 0.12	100.00 ± 0.00	100.00 ± 0.00
Client 5	96.56 ± 0.30	99.00 ± 0.15	99.06 ± 0.06	97.27 ± 0.22	99.39 ± 0.03	100.00 ± 0.00
Client 6	97.14 ± 1.10	97.49 ± 1.68	99.96 ± 0.03	98.44 ± 0.22	100.00 ± 0.00	100.00 ± 0.00
Client 7	95.20 ± 1.43	97.71 ± 1.35	98.73 ± 0.12	94.55 ± 1.48	99.31 ± 0.09	100.00 ± 0.00
Client 8	96.40 ± 0.35	98.19 ± 0.18	98.35 ± 0.07	96.95 ± 0.28	99.52 ± 0.11	100.00 ± 0.00
Client 9	97.17 ± 0.14	98.00 ± 0.07	98.32 ± 0.63	98.49 ± 0.11	97.61 ± 1.02	99.93 ± 0.05
Average	96.52 ± 1.03	97.92 ± 0.61	99.04 ± 0.16	97.32 ± 0.33	99.07 ± 0.29	**99.99 ± 0.01**
			N-BaIoT for the 9 devices ($K = 9$)			
Client 1	92.23 ± 1.48	94.41 ± 0.94	96.28 ± 0.74	93.15 ± 1.98	95.78 ± 0.84	97.58 ± 0.54
Client 2	86.12 ± 2.50	89.79 ± 1.75	92.40 ± 2.25	87.80 ± 2.91	90.33 ± 1.75	93.10 ± 1.10
Client 3	92.11 ± 1.66	94.30 ± 1.16	96.55 ± 0.83	92.20 ± 1.66	94.97 ± 1.22	97.60 ± 0.33
Client 4	97.89 ± 0.50	98.61 ± 0.31	99.35 ± 0.19	97.95 ± 0.83	98.90 ± 0.15	99.95 ± 0.05
Client 5	94.64 ± 1.30	95.59 ± 0.91	96.70 ± 0.65	94.75 ± 1.30	96.95 ± 0.91	98.80 ± 0.35
Client 6	96.27 ± 0.95	97.12 ± 0.67	98.33 ± 0.48	96.77 ± 0.95	98.35 ± 0.67	99.75 ± 0.08
Client 7	94.16 ± 1.44	96.92 ± 1.01	98.10 ± 0.72	94.60 ± 1.44	96.35 ± 1.01	97.90 ± 0.42
Client 8	93.88 ± 1.52	95.55 ± 0.98	96.80 ± 0.76	93.95 ± 1.52	95.05 ± 0.93	98.90 ± 0.36
Client 9	89.90 ± 2.19	91.10 ± 1.53	93.85 ± 1.40	90.15 ± 2.19	93.30 ± 1.53	96.05 ± 0.85
Average	93.02 ± 1.55	94.82 ± 1.03	96.48 ± 0.89	93.48 ± 1.64	95.55 ± 1.00	**97.74 ± 0.45**

4.2.3. Experimental Setting 3: Ablation Study

To comprehensively evaluate G-PFL-ID's robustness across different operational configurations, we conduct an ablation study that systematically varies two key parameters: (1) the degree of non-IIDness through the Dirichlet concentration parameter $\alpha \in \{0.1, 0.5, \infty\}$, and (2) the federation size $K \in \{10, 15, 20\}$. Table 11 reports the per-client AUC-ROC for the complete G-PFL-ID framework (with personalization heads) across these configurations for both GCN-DeepSVDD and GAE-DeepSVDD backbones on the IoT-23 dataset.

The ablation study shows that performance improves as data distributions become more IID (i.e increasing α) for a fixed federation size. With $K = 10$, GCN-DeepSVDD average AUC-ROC increases from 97.00% at $\alpha = 0.1$ to 99.04% at $\alpha = \infty$, while GAE-DeepSVDD shows a similar progression from 97.21% to 99.99%. This pattern holds across all K values, confirming that IID data facilitates global model convergence. However, even under extreme non-IID conditions ($\alpha = 0.1$), both variants maintain strong performance (above 94% for $K \leq 15$), demonstrating the framework's resilience to distributional skew.

The study also shows that increasing federation size (more clients with less data each) moderately impacts performance, particularly under high non-IIDness. For $\alpha = 0.1$, GAE-DeepSVDD average AUC-ROC slightly decreases by about as the number of clients increases from $K = 10$ to $K = 20$, particularly from $K = 10$ to $K = 15$. It then slightly recovers at $K = 20$ showing that generally as the client increases, the personalization stage becomes more critical because clients with very limited data benefit less from global aggregation but can still achieve reasonable performance through local adaptation.

The performance difference of the hybrid GAE-DeepSVDD over the pure GCN variant narrows down as the data becomes more IID across all configurations, suggesting that the reconstruction objective in GAE provides valuable regularization when global consensus is difficult to achieve. The hybrid architecture also exhibits lower performance variance, with standard deviations consistently 15-30% smaller than those of GCN-DeepSVDD.

Table 11. G-PFL-ID per-client AUC-ROC (%) results under different non-IID and number of clients settings.

Client ID	GCN-DeepSVDD			GAE-DeepSVDD		
	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = \infty$	$\alpha = 0.1$	$\alpha = 0.5$	$\alpha = \infty$
			10 Clients ($K = 10$)			
Client 0	95.97 ± 1.05	96.28 ± 0.62	99.95 ± 0.01	96.58 ± 0.58	99.06 ± 0.79	100.00 ± 0.00
Client 1	96.68 ± 0.47	97.45 ± 0.73	98.19 ± 0.09	96.49 ± 0.46	99.66 ± 0.17	99.98 ± 0.01
Client 2	94.30 ± 1.20	96.02 ± 0.56	99.49 ± 0.15	94.51 ± 0.98	99.49 ± 0.11	100.00 ± 0.00
Client 3	97.43 ± 0.59	98.26 ± 0.38	98.36 ± 0.46	97.64 ± 0.42	100.00 ± 0.00	99.97 ± 0.01
Client 4	99.08 ± 0.33	94.82 ± 0.46	100.00 ± 0.00	99.29 ± 0.09	98.93 ± 1.06	100.00 ± 0.00
Client 5	98.52 ± 0.34	95.78 ± 0.39	99.06 ± 0.06	98.73 ± 0.25	99.14 ± 0.33	100.00 ± 0.00
Client 6	99.48 ± 0.03	98.96 ± 0.07	99.96 ± 0.03	99.69 ± 0.07	100.00 ± 0.00	100.00 ± 0.00
Client 7	95.78 ± 0.91	97.49 ± 0.40	98.73 ± 0.12	94.99 ± 1.19	99.86 ± 0.09	100.00 ± 0.00
Client 8	93.22 ± 2.03	98.05 ± 0.32	98.35 ± 0.07	94.43 ± 0.79	99.46 ± 0.34	100.00 ± 0.00
Client 9	99.56 ± 0.07	99.38 ± 0.07	98.32 ± 0.63	99.77 ± 0.05	100.00 ± 0.00	99.93 ± 0.05
Average	97.00 ± 0.70	97.25 ± 0.40	99.04 ± 0.16	97.21 ± 0.49	99.56 ± 0.29	**99.99 ± 0.01**
			15 Clients ($K = 15$)			
Client 0	95.15 ± 0.62	96.89 ± 0.98	99.19 ± 0.38	96.13 ± 1.25	98.76 ± 0.45	100.00 ± 0.00
Client 1	94.82 ± 1.05	95.74 ± 1.24	99.04 ± 0.76	94.87 ± 1.68	97.92 ± 0.89	99.79 ± 0.09
Client 2	98.47 ± 0.34	97.28 ± 0.76	99.21 ± 0.24	96.42 ± 0.92	99.13 ± 0.32	100.00 ± 0.00
Client 3	91.19 ± 1.98	94.82 ± 2.85	96.89 ± 0.85	93.56 ± 2.14	97.14 ± 1.12	99.34 ± 0.12
Client 4	96.95 ± 0.62	98.63 ± 0.61	100.00 ± 0.00	97.88 ± 0.74	99.41 ± 0.25	100.00 ± 0.00
Client 5	91.72 ± 3.22	94.15 ± 2.03	96.45 ± 0.81	92.94 ± 2.41	96.57 ± 1.34	98.97 ± 0.34
Client 6	97.68 ± 1.09	98.52 ± 0.91	99.18 ± 0.41	95.67 ± 1.09	98.49 ± 0.52	99.89 ± 0.02
Client 7	97.02 ± 0.42	98.09 ± 0.81	99.93 ± 0.03	97.21 ± 0.97	98.98 ± 0.38	100.00 ± 0.00
Client 8	91.24 ± 1.41	95.37 ± 0.47	97.07 ± 0.47	94.52 ± 1.87	97.68 ± 0.97	99.68 ± 0.27
Client 9	97.98 ± 0.34	98.02 ± 0.51	100.00 ± 0.00	98.35 ± 0.61	99.72 ± 0.18	100.00 ± 0.00
Client 10	90.56 ± 2.78	94.96 ± 2.52	97.90 ± 0.47	93.87 ± 2.03	97.35 ± 1.05	99.85 ± 0.05
Client 11	96.63 ± 0.54	97.45 ± 0.68	99.95 ± 0.02	97.67 ± 0.82	99.26 ± 0.29	100.00 ± 0.00
Client 12	91.96 ± 2.12	96.24 ± 0.35	99.37 ± 0.36	95.28 ± 1.55	98.21 ± 0.72	99.81 ± 0.00
Client 13	98.21 ± 0.13	99.16 ± 0.56	100.00 ± 0.00	99.12 ± 0.68	99.58 ± 0.21	100.00 ± 0.00
Client 14	92.88 ± 2.63	95.03 ± 1.79	98.00 ± 0.79	93.79 ± 2.23	97.02 ± 1.18	99.41 ± 0.18
Average	94.83 ± 1.29	96.69 ± 1.14	98.81 ± 0.37	95.82 ± 1.40	98.35 ± 0.66	**99.78 ± 0.07**
			20 Clients ($K = 20$)			
Client 0	97.47 ± 0.45	98.24 ± 0.57	99.20 ± 0.37	96.95 ± 0.88	99.62 ± 0.12	100.00 ± 0.00
Client 1	94.18 ± 3.38	95.78 ± 2.21	98.78 ± 0.11	97.43 ± 0.67	99.34 ± 0.18	99.98 ± 0.00
Client 2	96.84 ± 0.58	98.05 ± 0.18	99.00 ± 0.52	97.26 ± 1.42	98.47 ± 0.68	99.49 ± 0.16
Client 3	93.37 ± 4.82	95.91 ± 2.58	97.11 ± 0.58	95.58 ± 3.12	97.73 ± 1.24	99.13 ± 0.42
Client 4	98.76 ± 0.62	99.82 ± 0.09	100.00 ± 0.00	98.84 ± 0.15	99.12 ± 0.28	100.00 ± 0.00
Client 5	95.56 ± 0.32	96.13 ± 1.89	97.88 ± 0.29	96.72 ± 3.02	98.02 ± 1.08	99.92 ± 0.00
Client 6	97.25 ± 1.25	98.67 ± 0.39	99.17 ± 0.09	98.63 ± 0.73	99.09 ± 0.02	100.00 ± 0.00
Client 7	97.39 ± 1.08	98.48 ± 0.72	99.08 ± 0.12	97.81 ± 1.28	98.86 ± 0.35	99.98 ± 0.00
Client 8	92.94 ± 2.82	94.52 ± 0.85	98.15 ± 0.15	93.27 ± 2.24	96.98 ± 1.18	98.98 ± 0.16
Client 9	98.27 ± 0.48	99.15 ± 0.18	100.00 ± 0.00	98.59 ± 0.98	99.45 ± 0.38	100.00 ± 0.00
Client 10	96.79 ± 0.62	98.26 ± 0.42	99.72 ± 0.14	97.08 ± 0.92	98.89 ± 0.48	99.89 ± 0.06
Client 11	93.58 ± 0.66	96.71 ± 0.86	99.01 ± 0.16	95.97 ± 1.12	99.03 ± 0.44	100.00 ± 0.00
Client 12	97.36 ± 1.68	98.89 ± 0.52	99.90 ± 0.01	96.74 ± 2.11	99.25 ± 0.05	100.00 ± 0.00
Client 13	98.15 ± 0.52	99.03 ± 0.02	100.00 ± 0.00	96.42 ± 1.02	99.31 ± 0.40	100.00 ± 0.00
Client 14	95.52 ± 3.25	97.94 ± 1.18	99.86 ± 0.06	97.81 ± 0.58	99.17 ± 0.12	100.00 ± 0.00
Client 15	97.07 ± 0.05	99.19 ± 0.28	99.96 ± 0.02	96.98 ± 1.55	99.28 ± 0.05	100.00 ± 0.00
Client 16	95.57 ± 2.38	98.42 ± 0.51	99.22 ± 0.01	94.93 ± 1.85	97.84 ± 0.89	99.68 ± 0.10
Client 17	92.68 ± 1.72	96.59 ± 0.96	98.89 ± 0.16	96.05 ± 1.21	98.94 ± 0.52	99.91 ± 0.03
Client 18	98.21 ± 0.08	99.75 ± 0.22	100.00 ± 0.00	97.52 ± 1.38	99.86 ± 0.14	100.00 ± 0.00
Client 19	96.94 ± 0.58	97.87 ± 0.84	98.67 ± 0.04	96.28 ± 1.06	99.18 ± 0.42	100.00 ± 0.00
Average	96.20 ± 1.37	97.87 ± 0.77	99.18 ± 0.14	96.84 ± 1.36	98.87 ± 0.45	**99.85 ± 0.05**

These findings validate several design choices in G-PFL-ID. The FedReg regularizer (FedProx + variance penalty) proves essential for maintaining stability across diverse non-IID levels, as evidenced by the consistent performance improvements across α values. Personalization heads effectively mitigate the impact of increased federation size, allowing clients to adapt the global model to their specific data distributions. The GAE backbone's dual reconstruction-compactness objective provides more robust representations than the pure one-class objective of GCN-DeepSVDD, especially when clients have limited or highly skewed data.

5. Conclusions

In this paper, we proposed G-PFL-ID, a graph-driven personalized federated learning framework for unsupervised intrusion detection in non-IID IoT systems. The framework leverages graph neural networks (GCN and GAE) combined with a DeepSVDD objective to learn compact representations of normal network behavior. To address the challenges of non-IID data, we introduced FedReg, a regularization method that includes a proximal term and an embedding variance penalty. Furthermore, we enabled personalization through client-specific heads to adapt the global model to local data distributions.

We evaluated G-PFL-ID on two IoT intrusion datasets under diverse heterogeneity scenarios. On IoT-23, we used Dirichlet-based partitioning to simulate varying degrees of non-IIDness ($\alpha \in \{0.1, 0.5, \infty\}$) and federation sizes ($K \in \{10, 15, 20\}$), while N-BaIoT provided natural device-based partitioning with inherent heterogeneity (9.0:1 quantity skew, Gini coefficient 0.37). The results demonstrate that our approach significantly outperforms baseline methods, including classical one-class algorithms, federated MLP models, and recent graph-based federated anomaly detectors (e.g., LG-FGAD), in terms of AUROC, AUPR, and detection rates at low false positive rates. The hybrid GAE-DeepSVDD instantiation delivered the strongest global detection performance (99.46% AUROC on IoT-23, 97.74% on N-BaIoT) and the best per-client robustness across both artificial and natural partitioning schemes, while the GCN-DeepSVDD variant served as a principled baseline to assess the role of inductive bias.

Ablation studies confirm that (i) FedProx reduces parameter divergence under quantity skew, (ii) the variance penalty prevents embedding collapse on low-data clients and improves downstream AUROC, and (iii) lightweight personalization recovers client-level thresholds, especially for low-data clients under extreme non-IID settings. On N-BaIoT, despite extreme natural heterogeneity, G-PFL-ID achieved 97.74% average AUROC for GAE-DeepSVDD after personalization, with low-data devices showing the largest relative improvements (up to 6 percentage points).

We highlight three practical core insights for deploying federated graph anomaly detectors in IoT networks. First, combining reconstruction and one-class compactness provides complementary signals that improve resilience to heterogeneity across both artificial and natural partitioning. Second, simple, aggregation-aware regularizers (FedReg) are effective and computationally inexpensive, which is important for constrained edge devices. Third, personalization via small heads is an efficient way to produce strong per-client performance while keeping the backbone shared, with particular value in real-world deployments where device heterogeneity is intrinsic rather than artificially induced.

Limitations include reliance on reasonably representative benign data for one-class training, sensitivity to extreme concept drift, and the need to evaluate privacy/robustness under adversarial updates (e.g., targeted poisoning). Future work will extend G-PFL-ID to (a) adversarially robust aggregation and certified defenses, (b) continual calibration for concept drift, and (c) evaluation over wider operational datasets and live deployments. We release our code, model definitions, and partitioning scripts to enable reproducible follow-up research.

Author Contributions: Conceptualization, D.A.O.; methodology, D.A.O.; software, D.A.O.; validation, D.A.O.; formal analysis, D.A.O.; investigation, D.A.O.; resources, D.A.O., M.S. and E.M.; data curation, D.A.O.; writing—original draft preparation, D.A.O.; writing—review and editing, D.A.O., A.I., O.A.-A., O.E., S.G.T., M.S. and E.M.; visualization, D.A.O.; supervision, M.S. and E.M.; project administration, M.S. and E.M.; funding acquisition, M.S. and E.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: Code implementation can be found at G-PFL-ID: Graph Personalized Federated Learning Intrusion Detection (https://github.com/danieloladele7/g-pfl-ids, accessed on 23 November 2025). The IoT-23 dataset is available at Aposemat IoT-23 (https://www.stratosphereips.org/datasets-iot23, accessed on 23 November 2025). The N-BaIoT dataset is available at UCI Machine Learning Repository (https://archive.ics.uci.edu/ml/datasets/detection_of_IoT_botnet_attacks_N_BaIoT, accessed on 3 January 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

G-PFL-ID	Graph-Driven Personalized Federated Learning for Intrusion Detection
non-IID	Non-Independent and Identically Distributed
GAE/GCN	Global Graph Autoencoder or Graph Convolutional Network
DoS/DDoS	Denial of Service/Distributed DoS
IOT	Internet of Things
IDS/AD	Intrusion Detection System/Anomaly Detection
ML/FL	Machine Learning/Federated Learning
AUROC	Area Under the Receiver Operating Characteristic Curve
AUPR	Area Under the Precision–Recall Curve
FPR/TPR	False Positive Rates/True Positive Rates

References

1. Oladele, D.A.; Didam Markus, E.; Abu-Mahfouz, A.M. Adaptability of Assistive Mobility Devices and the Role of the Internet of Medical Things: Comprehensive Review. *JMIR Rehabil. Assist. Technol.* **2021**, *8*, e29610. [CrossRef] [PubMed]
2. Shehu Yalli, J.; Hilmi Hasan, M.; Abubakar Badawi, A. Internet of Things (IoT): Origins, Embedded Technologies, Smart Applications, and Its Growth in the Last Decade. *IEEE Access* **2024**, *12*, 91357–91382. [CrossRef]
3. Ahmed, S.F.; Alam, M.S.B.; Hoque, M.; Lameesa, A.; Afrin, S.; Farah, T.; Kabir, M.; Shafiullah, G.M.; Muyeen, S.M. Industrial Internet of Things enabled technologies, challenges, and future directions. *Comput. Electr. Eng.* **2023**, *110*, 108847. [CrossRef]
4. Oladele, D.A.; Didam Markus, E.; Abu-Mahfouz, A.M. BEV-CAM3D: A Unified Bird's-Eye View Architecture for Autonomous Driving with Monocular Cameras and 3D Point Clouds. *AI* **2025**, *6*, 82. [CrossRef]
5. Oladele, D.A.; Didam Markus, E.; Abu-Mahfouz, A.M. FASTSeg3D: A Fast, Efficient, and Adaptive Ground Filtering Algorithm for 3D Point Clouds in Mobile Sensing Applications. *AI* **2025**, *6*, 97. [CrossRef]
6. Pech, M.; Vrchota, J.; Bednář, J. Predictive Maintenance and Intelligent Sensors in Smart Factory: Review. *Sensors* **2021**, *21*, 1470. [CrossRef]
7. Antonakakis, M.; April, T.; Bailey, M.; Bernhard, M.; Bursztein, E.; Cochran, J.; Durumeric, Z.; Halderman, J.A.; Invernizzi, L.; Kallitsis, M.; et al. Understanding the Mirai botnet. In *Proceedings of the 26th USENIX Conference on Security Symposium, Vancouver, BC, Canada, 16–18 August 2017*; USENIX Association: Berkeley, CA, USA, 2017; pp. 1093–1110.
8. Boyarchuk, O.; Mariani, S.; Ortolani, S.; Vigna, G. Keeping Up with the Emotets: Tracking a Multi-infrastructure Botnet. *Digit. Threat.* **2023**, *4*, 41. [CrossRef]
9. Tanwar, G.S.; Goar, V. Tools, Techniques & Analysis of Botnet. In *Proceedings of the International Conference on Information and Communication Technology for Competitive Strategies*; Association for Computing Machinery: New York, NY, USA, 2014; pp. 92–96. [CrossRef]
10. Rizvi, S.; Orr, R.J.; Cox, A.; Ashokkumar, P.; Rizvi, M.R. Identifying the attack surface for IoT network. *Internet Things* **2020**, *9*, 100162. [CrossRef]
11. Hnamte, V.; Hussain, J. Enhancing security in Software-Defined Networks: An approach to efficient ARP spoofing attacks detection and mitigation. *Telemat. Inform. Rep.* **2024**, *14*, 100129. [CrossRef]
12. Kambourakis, G.; Kolias, C.; Stavrou, A. The Mirai botnet and the IoT Zombie Armies. In *Proceedings of the MILCOM 2017 IEEE Military Communications Conference (MILCOM), Baltimore, MD, USA, 23–25 October 2017*; IEEE: Piscataway, NJ, USA, 2017; pp. 267–272. [CrossRef]
13. Paquet, C. Implementing Cisco IOS Network Security (IINS). In *PART II: Protecting the Network Infrastructure*; Cisco Press: Indianapolis, IN, USA, 2009; Chapter 3–6. pp. 111–300.
14. Paquet, C. Implementing Cisco IOS Network Security (IINS). In *PART III: Threat Control and Containment*; Cisco Press: Indianapolis, IN, USA, 2009; Chapter 7–11. pp. 305–520.

15. Lekkala, S.; Gurijala, P. Designing Secure Network Architectures. In *Security and Privacy for Modern Networks: Strategies and Insights for Safeguarding Digital Infrastructures*; Apress: Berkeley, CA, USA, 2024; Chapter 8, pp. 75–86. [CrossRef]
16. Lekkala, S.; Gurijala, P. Securing Networks with SDN and SD-WAN. In *Security and Privacy for Modern Networks: Strategies and Insights for Safeguarding Digital Infrastructures*; Apress: Berkeley, CA, USA, 2024; Chapter 12, pp. 121–131. [CrossRef]
17. Lekkala, S.; Gurijala, P. Proactive Intrusion Detection and Network Surveillance. In *Security and Privacy for Modern Networks: Strategies and Insights for Safeguarding Digital Infrastructures*; Apress: Berkeley, CA, USA, 2024; Chapter 10, pp. 99–108. [CrossRef]
18. Abdulganiyu, O.H.; Tchakoucht, T.A.; Saheed, Y.K. A systematic literature review for network intrusion detection system (IDS). *Int. J. Inf. Secur.* **2023**, *22*, 1125–1162. [CrossRef]
19. Roesch, M. Snort–Lightweight Intrusion Detection for Networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99), Seattle, WA, USA, 7–12 November 1999*; USENIX Association: Berkeley, CA, USA, 1999; pp. 229–238.
20. Khraisat, A.; Gondal, I.; Vamplew, P.; Kamruzzaman, J. Survey of intrusion detection systems: Techniques, datasets and challenges. *Cybersecurity* **2019**, *2*, 20. [CrossRef]
21. Chandola, V.; Banerjee, A.; Kumar, V. Anomaly detection: A survey. *ACM Comput. Surv.* **2009**, *41*, 15. [CrossRef]
22. Ferrag, M.A.; Maglaras, L.; Moschoyiannis, S.; Janicke, H. Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study. *J. Inf. Secur. Appl.* **2020**, *50*, 102419. [CrossRef]
23. Guerra, J.L.; Catania, C.; Veas, E. Datasets are not enough: Challenges in labeling network traffic. *Comput. Secur.* **2022**, *120*, 102810. [CrossRef]
24. Alotaibi, B. A Survey on Industrial Internet of Things Security: Requirements, Attacks, AI-Based Solutions, and Edge Computing Opportunities. *Sensors* **2023**, *23*, 7470. [CrossRef]
25. Kikissagbe, B.R.; Adda, M. Machine Learning-Based Intrusion Detection Methods in IoT Systems: A Comprehensive Review. *Electronics* **2024**, *13*, 3601. [CrossRef]
26. Sakurada, M.; Yairi, T. Anomaly Detection Using Autoencoders with Nonlinear Dimensionality Reduction. In *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis, Gold Coast, QLD, Australia, 2 December 2014*; Association for Computing Machinery: New York, NY, USA, 2014; pp. 4–11. [CrossRef]
27. Ruff, L.; Vandermeulen, R.; Goernitz, N.; Deecke, L.; Siddiqui, S.A.; Binder, A.; Müller, E.; Kloft, M. Deep One-Class Classification. In *Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, 10–15 July 2018*; Dy, J., Krause, A., Eds.; PMLR: Cambridge, MA, USA, 2018; Volume 80, pp. 4393–4402. Available online: https://proceedings.mlr.press/v80/ruff18a.html (accessed on 5 December 2025).
28. Kipf, T.N.; Welling, M. Variational Graph Auto-Encoders. In *Proceedings of the Bayesian Deep Learning Workshop, NeurIPS 2016*; NeurIPS Foundation: Barcelona, Spain, 2016; pp. 1–3. Available online: https://bayesiandeeplearning.org/2016/ (accessed on 5 December 2025).
29. Jiang, B.; Zhang, Z.; Lin, D.; Tang, J.; Luo, B. Semi-Supervised Learning With Graph Learning-Convolutional Networks. In Proceedings of the 2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Long Beach, CA, USA, 15–20 June 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 11305–11312. [CrossRef]
30. Kipf, T.N.; Welling, M. Semi-Supervised Classification with Graph Convolutional Networks. *arXiv* **2016**, arXiv:1609.02907. [CrossRef]
31. Rahman, S.A.; Tout, H.; Talhi, C.; Mourad, A. Internet of Things Intrusion Detection: Centralized, On-Device, or Federated Learning? *IEEE Netw.* **2020**, *34*, 310–317. [CrossRef]
32. Sultana, N.; Chilamkurti, N.; Peng, W.; Alhadad, R. Survey on SDN based network intrusion detection system using machine learning approaches. *Peer-to-Peer Netw. Appl.* **2019**, *12*, 493–501. [CrossRef]
33. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; Agüera y Arcas, B. Communication-Efficient Learning of Deep Networks from Decentralized Data . In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS), Fort Lauderdale, FL, USA, 20–22 April 2017*; Singh, A., Zhu, J., Eds.; PMLR: Cambridge, MA, USA , 2017; Volume 54, pp. 1273–1282. Available online: https://proceedings.mlr.press/v54/mcmahan17a.html (accessed on 5 December 2025).
34. Kairouz, P.; McMahan, H.B.; Avent, B.; Bellet, A.; Bennis, M.; Nitin Bhagoji, A.; Bonawitz, K.; Charles, Z.; Cormode, G.; Cummings, R.; et al. Advances and Open Problems in Federated Learning. *arXiv* **2019**, arXiv:1912.04977. [CrossRef]
35. Tang, Z.; Hu, H.; Xu, C. A federated learning method for network intrusion detection. *Concurr. Comput. Pract. Exp.* **2022**, *34*, e6812. [CrossRef]
36. Friha, O.; Ferrag, M.A.; Shu, L.; Maglaras, L.; Choo, K.R.; Nafaa, M. FELIDS: Federated learning-based intrusion detection system for agricultural Internet of Things. *J. Parallel Distrib. Comput.* **2022**, *165*, 17–31. [CrossRef]
37. Mazid, A.; Kirmani, S.; Manaullah.; Yadav, M. FL-IDPP: A Federated Learning Based Intrusion Detection Approach with Privacy Preservation. *Trans. Emerg. Telecommun. Technol.* **2025**, *36*, e70039. [CrossRef]
38. McMahan, H.B.; Moore, E.; Ramage, D.; Hampson, S.; Agüera y Arcas, B. Communication-Efficient Learning of Deep Networks from Decentralized Data. *arXiv* **2023**, arXiv:1602.05629. [CrossRef]
39. Zhao, Y.; Li, M.; Lai, L.; Suda, N.; Civin, D.; Chandra, V. Federated Learning with Non-IID Data. *arXiv* **2018**. [CrossRef]

40. Campos, E.M.; Saura, P.F.; González-Vidal, A.; Hernández-Ramos, J.L.; Bernabé, J.B.; Baldini, G.; Skarmeta, A. Evaluating Federated Learning for intrusion detection in Internet of Things: Review and challenges. *Comput. Netw.* **2022**, *203*, 108661. [CrossRef]
41. Agrawal, S.; Sarkar, S.; Aouedi, O.; Yenduri, G.; Piamrat, K.; Alazab, M.; Bhattacharya, S.; Maddikunta, P.K.R.; Gadekallu, T.R. Federated Learning for intrusion detection system: Concepts, challenges and future directions. *Comput. Commun.* **2022**, *195*, 346–361. [CrossRef]
42. Khraisat, A.; Alazab, A.; Singh, S.; Jan, T.; Jr., A.G. Survey on Federated Learning for Intrusion Detection System: Concept, Architectures, Aggregation Strategies, Challenges, and Future Directions. *ACM Comput. Surv.* **2024**, *57*, 7. [CrossRef]
43. Hernandez-Ramos, J.L.; Karopoulos, G.; Chatzoglou, E.; Kouliaridis, V.; Marmol, E.; Gonzalez-Vidal, A.; Kambourakis, G. Intrusion Detection Based on Federated Learning: A Systematic Review. *ACM Comput. Surv.* **2025**, *57*, 309. [CrossRef]
44. Lu, Z.; Pan, H.; Dai, Y.; Si, X.; Zhang, Y. Federated Learning with Non-IID Data: A Survey. *IEEE Internet Things J.* **2024**, *11*, 19188–19209. [CrossRef]
45. Ma, X.; Zhu, J.; Lin, Z.; Chen, S.; Qin, Y. A State-of-the-Art Survey on Solving Non-IID Data in Federated Learning. *Future Gener. Comput. Syst.* **2022**, *135*, 244–258. [CrossRef]
46. Xie, H.; Ma, J.; Xiong, L.; Yang, C. Federated Graph Classification over Non-IID Graphs . *Adv. Neural Inf. Process. Syst.* **2021**, *34*, 18839–18852.
47. Yao, Y.; Jin, W.; Ravi, S.; Joe-Wong, C. FedGCN: Convergence-Communication Tradeoffs in Federated Training of Graph Convolutional Networks. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 79748–79760.
48. Li, T.; Sahu, A.K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; Smith, V. Federated Optimization in Heterogeneous Networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 429–450.
49. Bhavsar, M.H.; Bekele, Y.B.; Roy, K.; Kelly, J.C.; Limbrick, D. FL-IDS: Federated Learning-Based Intrusion Detection System Using Edge Devices for Transportation IoT. *IEEE Access* **2024**, *12*, 52215–52226. [CrossRef]
50. Nguyen, Q.H.; Hore, S.; Shah, A.; Le, T.; Bastian, N.D. FedNIDS: A Federated Learning Framework for Packet-Based Network Intrusion Detection System. *Digit. Threat.* **2025**, *6*, 4. [CrossRef]
51. Althunayyan, M.; Javed, A.; Rana, O. A robust multi-stage intrusion detection system for in-vehicle network security using hierarchical federated learning. *Veh. Commun.* **2024**, *49*, 100837. [CrossRef]
52. Huang, K.; Xian, R.; Xian, M.; Wang, H.; Ni, L. A comprehensive intrusion detection method for the Internet of Vehicles based on federated learning architecture. *Comput. Secur.* **2024**, *147*, 104067. [CrossRef]
53. Shao, J.; Zeng, G.; Lu, K.; Geng, G.; Weng, J. Automated federated learning for intrusion detection of industrial control systems based on evolutionary neural architecture search. *Comput. Secur.* **2024**, *143*, 103910. [CrossRef]
54. Praharaj, L.; Gupta, D.; Gupta, M. Efficient federated transfer learning-based network anomaly detection for cooperative smart farming infrastructure. *Smart Agric. Technol.* **2025**, *10*, 100727. [CrossRef]
55. Popli, M.S.; Singh, R.P.; Popli, N.K.; Mamun, M. A Federated Learning Framework for Enhanced Data Security and Cyber Intrusion Detection in Distributed Network of Underwater Drones. *IEEE Access* **2025**, *13*, 12634–12646. [CrossRef]
56. Jin, Z.; Zhou, J.; Li, B.; Wu, X.; Duan, C. FL-IIDS: A novel federated learning-based incremental intrusion detection system. *Future Gener. Comput. Syst.* **2024**, *151*, 57–70. [CrossRef]
57. Singh, G.; Sood, K.; Rajalakshmi, P.; Nguyen, D.D.N.; Xiang, Y. Evaluating Federated Learning-Based Intrusion Detection Scheme for Next Generation Networks. *IEEE Trans. Netw. Serv. Manag.* **2024**, *21*, 4816–4829. [CrossRef]
58. Bank, D.; Koenigstein, N.; Giryes, R. Autoencoders. *arXiv* **2021**. [CrossRef]
59. Kingma, D.P.; Welling, M. An Introduction to Variational Autoencoders. *Found. Trends Mach. Learn.* **2019**, *12*, 307–392. [CrossRef]
60. Yepmo, V.; Smits, G.; Lesot, M.J.; Pivert, O. Leveraging an Isolation Forest to Anomaly Detection and Data Clustering. *Data Knowl. Eng.* **2024**, *151*, 102302. [CrossRef]
61. Dong, B.; Chen, D.; Wu, Y.; Tang, S.; Zhuang, Y. FADngs: Federated Learning for Anomaly Detection. *IEEE Trans. Neural Networks Learn. Syst.* **2025**, *36*, 2578–2592. [CrossRef]
62. Nguyen, V.T.; Beuran, R. FedMSE: Semi-supervised federated learning approach for IoT network intrusion detection. *Comput. Secur.* **2025**, *151*, 104337. [CrossRef]
63. Zhang, A.; Zhao, P.; Lu, W.; Zhou, Y.; Zhang, W.; Zhang, G. Mitigating Poisoning Attacks in Federated Learning Through Deep One-Class Classification. *IEEE Trans. Cogn. Commun. Netw.* **2025**, *12*, 545–558. [CrossRef]
64. Lu, Y.; Yang, T.; Zhao, C.; Chen, W.; Zeng, R. A swarm anomaly detection model for IoT UAVs based on a multi-modal denoising autoencoder and federated learning. *Comput. Ind. Eng.* **2024**, *196*, 110454. [CrossRef]
65. Shrestha, R.; Mohammadi, M.; Sinaei, S.; Salcines, A.; Pampliega, D.; Clemente, R.; Sanz, A.L.; Nowroozi, E.; Lindgren, A. Anomaly detection based on LSTM and autoencoders using federated learning in smart electric grid. *J. Parallel Distrib. Comput.* **2024**, *193*, 104951. [CrossRef]
66. He, Y.; Ding, X.; Tang, Y.; Guan, J.; Zhou, S. Unsupervised Multivariate Time Series Anomaly Detection by Feature Decoupling in Federated Learning Scenarios. *IEEE Trans. Artif. Intell.* **2025**, *6*, 2013–2026. [CrossRef]

67. Liao, X.; Liu, W.; Chen, C.; Zhou, P.; Yu, F.; Zhu, H.; Yao, B.; Wang, T.; Zheng, X.; Tan, Y. Rethinking the Representation in Federated Unsupervised Learning with Non-IID Data. In *Proceedings of the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 16–22 June 2024*; IEEE: Piscataway, NJ, USA, 2024; pp. 22841–22850. [CrossRef]
68. Wu, Z.; Pan, S.; Chen, F.; Long, G.; Zhang, C.; Yu, P.S. A Comprehensive Survey on Graph Neural Networks. *IEEE Trans. Neural Networks Learn. Syst.* **2021**, *32*, 4–24. [CrossRef] [PubMed]
69. Yuan, H.; Yu, H.; Gui, S.; Ji, S. Explainability in Graph Neural Networks: A Taxonomic Survey. *IEEE Trans. Pattern Anal. Mach. Intell.* **2023**, *45*, 5782–5799. [CrossRef]
70. Wang, J.; Liang, J.; Yao, K.; Liang, J.; Wang, D. Graph convolutional autoencoders with co-learning of graph structure and node attributes. *Pattern Recognit.* **2022**, *121*, 108215. [CrossRef]
71. Roy, A.; Shu, J.; Li, J.; Yang, C.; Elshocht, O.; Smeets, J.; Li, P. GAD-NR: Graph Anomaly Detection via Neighborhood Reconstruction. In *Proceedings of the 17th ACM International Conference on Web Search and Data Mining, WSDM '24, Merida, Mexico, 4–8 March 2024*; Association for Computing Machinery: New York, NY, USA, 2024; pp. 576–585. [CrossRef]
72. Wu, N.; Zhao, Y.; Dong, H.; Xi, K.; Yu, W.; Wang, W. Federated Graph Anomaly Detection Through Contrastive Learning with Global Negative Pairs. *Proc. Aaai Conf. Artif. Intell.* **2025**, *39*, 21554–21562. [CrossRef]
73. Kong, X.; Zhang, W.; Wang, H.; Hou, M.; Chen, X.; Yan, X.; Das, S.K. Federated Graph Anomaly Detection via Contrastive Self-Supervised Learning. *IEEE Trans. Neural Netw. Learn. Syst.* **2025**, *36*, 7931–7944. [CrossRef]
74. Cai, J.; Zhang, Y.; Fan, J.; Ng, S.K. LG-FGAD: An Effective Federated Graph Anomaly Detection Framework. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence, IJCAI-24, Jeju, Republic of Korea, 3–9 August 2024*; Larson, K., Ed.; Main Track; International Joint Conferences on Artificial Intelligence Organization: Bremen, Germany, 2024; Volume 8, pp. 3760–3769. [CrossRef]
75. Xia, F.; Sun, K.; Yu, S.; Aziz, A.; Wan, L.; Pan, S.; Liu, H. Graph Learning: A Survey. *IEEE Trans. Artif. Intell.* **2021**, *2*, 109–127. [CrossRef]
76. Tax, D.M.J.; Duin, R.P.W. Support Vector Data Description. *Mach. Learn.* **2004**, *54*, 45–66. [CrossRef]
77. Zhou, Y.; Liang, X.; Zhang, W.; Zhang, L.; Song, X. VAE-based Deep SVDD for anomaly detection. *Neurocomputing* **2021**, *453*, 131–140. [CrossRef]
78. Yi, J.; Yoon, S. Patch SVDD: Patch-Level SVDD for Anomaly Detection and Segmentation. In *Proceedings of the Computer Vision—ACCV 2020, Kyoto, Japan, 30 November–4 December 2020*; Springer International Publishing: Berlin/Heidelberg, Germany, 2021; pp. 375–390.
79. Zhu, H.; Xu, J.; Liu, S.; Jin, Y. Federated learning on non-IID data: A survey. *Neurocomputing* **2021**, *465*, 371–390. [CrossRef]
80. Arivazhagan, M.G.; Aggarwal, V.; Singh, A.K.; Choudhary, S. Federated Learning with Personalization Layers. *arXiv* **2019**. [CrossRef]
81. Collins, L.; Hassani, H.; Mokhtari, A.; Shakkottai, S. Exploiting Shared Representations for Personalized Federated Learning. In *Proceedings of the 38th International Conference on Machine Learning, Virtual, 18–24 July 2021*; Meila, M., Zhang, T., Eds.; PMLR: Cambridge, MA, USA, 2021; Volume 139, pp. 2089–2099. Available online: https://proceedings.mlr.press/v139/collins21a.html (accessed on 5 December 2025).
82. Thein, T.T.; Shiraishi, Y.; Morii, M. Personalized federated learning-based intrusion detection system: Poisoning attack and defense. *Future Gener. Comput. Syst.* **2024**, *153*, 182–192. [CrossRef]
83. Li, T.; Hu, S.; Beirami, A.; Smith, V. Ditto: Fair and Robust Federated Learning Through Personalization. In *Proceedings of the 38th International Conference on Machine Learning, Virtual, 18–24 July 2021*; Meila, M., Zhang, T., Eds.; PMLR: Cambridge, MA, USA, 2021; Volume 139, pp. 6357–6368. Available online: https://proceedings.mlr.press/v139/li21h.html (accessed on 5 December 2025).
84. Fallah, A.; Mokhtari, A.; Ozdaglar, A. Personalized Federated Learning with Theoretical Guarantees: A Model-Agnostic Meta-Learning Approach . In *Proceedings of the Advances in Neural Information Processing Systems, Virtual, 6–12 December 2020*; Larochelle, H., Ranzato, M., Hadsell, R., Balcan, M., Lin, H., Eds.; Curran Associates, Inc.: Red Hook, NY, USA, 2020; Volume 33, pp. 3557–3568.
85. Yang, L.; Huang, J.; Lin, W.; Cao, J. Personalized Federated Learning on Non-IID Data via Group-based Meta-learning. *ACM Trans. Knowl. Discov. Data* **2023**, *17*, 49. [CrossRef]
86. Jiang, Y.; Konečný, J.; Rush, K.; Kannan, S. Improving Federated Learning Personalization via Model Agnostic Meta Learning. *arXiv* **2023**. http://arxiv.org/abs/1909.12488. [CrossRef]
87. de Cámara, X.S.; Flores, J.L.; Arellano, C.; Urbieta, A.; Zurutuza, U. Clustered federated learning architecture for network anomaly detection in large scale heterogeneous IoT networks. *Comput. Secur.* **2023**, *131*, 103299. [CrossRef]
88. Zhou, J.; Xie, H.; Yang, C. Graph Personalized Federated Learning via Client Network Learning. *Trans. Mach. Learn. Res.* **2025** . Available online: https://openreview.net/forum?id=pyTTR4pxkU (accessed on 5 December 2025).
89. Zhu, D.; Ma, Y.; Liu, Y. Anomaly Detection with Deep Graph Autoencoders on Attributed Networks. In *Proceedings of the 2020 IEEE Symposium on Computers and Communications (ISCC), Rennes, France, 7–10 July 2020*; IEEE: Piscataway, NJ, USA, 2020; pp. 1–6. [CrossRef]

90. Liu, R.; Xing, P.; Deng, Z.; Li, A.; Guan, C.; Yu, H. Federated Graph Neural Networks: Overview, Techniques, and Challenges. *IEEE Trans. Neural Networks Learn. Syst.* **2025**, *36*, 4279–4295. [CrossRef] [PubMed]
91. Ruff, L.; Vandermeulen, R.A.; Görnitz, N.; Binder, A.; Müller, E.; Müller, K.R.; Kloft, M. Deep Semi-Supervised Anomaly Detection. *arXiv* **2020**, arXiv:1906.02694. [CrossRef]
92. Nagarajan, V.; Andreassen, A.; Neyshabur, B. Understanding the failure modes of out-of-distribution generalization. *arXiv* **2021**, arXiv:2010.15775. [CrossRef]
93. Tan, A.Z.; Yu, H.; Cui, L.; Yang, Q. Towards Personalized Federated Learning. *IEEE Trans. Neural Networks Learn. Syst.* **2023**, *34*, 9587–9603. [CrossRef] [PubMed]
94. Beutel, D.J.; Topal, T.; Mathur, A.; Qiu, X.; Fernandez-Marques, J.; Gao, Y.; Sani, L.; Kwing, H.L.; Parcollet, T.; Gusmão, P.P.d.; et al. Flower: A Friendly Federated Learning Research Framework. *arXiv* **2020**, arXiv:2007.14390.
95. Paszke, A.; Gross, S.; Massa, F.; Lerer, A.; Bradbury, J.; Chanan, G.; Killeen, T.; Lin, Z.; Gimelshein, N.; Antiga, L.; et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Adv. Neural Inf. Process. Syst.* **2019**, *32*. [CrossRef]
96. Fey, M.; Lenssen, J.E. Fast Graph Representation Learning with PyTorch Geometric. *arXiv* **2019**. [CrossRef]
97. Meidan, Y.; Bohadana, M.; Mathov, Y.; Mirsky, Y.; Shabtai, A.; Breitenbacher, D.; Elovici, Y. N-BaIoT: Network-based Detection of IoT Botnet Attacks Using Deep Autoencoders. *IEEE Pervasive Comput.* **2018**, *17*, 12–22. [CrossRef]
98. Hsu, T.M.H.; Qi, H.; Brown, M. Measuring the Effects of Non-Identical Data Distribution for Federated Visual Classification. *arXiv* **2019**. [CrossRef]
99. Yurochkin, M.; Agarwal, M.; Ghosh, S.; Greenewald, K.; Hoang, T.N.; Khazaeni, Y. Bayesian Nonparametric Federated Learning of Neural Networks. *arXiv* **2019**. [CrossRef]
100. Fuglede, B.; Topsoe, F. Jensen-Shannon divergence and Hilbert space embedding. In *Proceedings of the International Symposium on Information Theory, 2004, ISIT 2004, Chicago, IL, USA, 27 June–2 July 2004*; IEEE: Piscataway, NJ, USA, 2004; p. 31. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.